



Universidade Estadual de Campinas
Instituto de Computação

Sandro Henrique Uliana Catabriga

Formulações de Programação Linear Inteira para o Problema da Compra Mínima

CAMPINAS
2025

Sandro Henrique Uliana Catabriga

**Formulações de Programação Linear Inteira para o Problema da
Compra Mínima**

Dissertação apresentada ao Instituto de
Computação da Universidade Estadual de
Campinas como parte dos requisitos para a
obtenção do título de Mestre em Ciência da
Computação.

Orientador: Prof. Dr. Rafael Crivellari Saliba Schouery

Este exemplar corresponde à versão final da
Dissertação defendida por Sandro Henrique
Uliana Catabriga e orientada pelo Prof. Dr.
Rafael Crivellari Saliba Schouery.

CAMPINAS
2025

Ficha catalográfica
Universidade Estadual de Campinas (UNICAMP)
Biblioteca do Instituto de Matemática, Estatística e Computação Científica
Ana Regina Machado - CRB 8/5467

C28f Catabriga, Sandro Henrique Uliana, 1999-
Formulações de programação linear inteira para o problema da compra
mínima / Sandro Henrique Uliana Catabriga. – Campinas, SP : [s.n.], 2025.

Orientador: Rafael Crivellari Saliba Schouery.
Dissertação (mestrado) – Universidade Estadual de Campinas
(UNICAMP), Instituto de Computação.

1. Precificação. 2. Programação inteira. 3. Otimização combinatória. I.
Schouery, Rafael Crivellari Saliba, 1986-. II. Universidade Estadual de
Campinas (UNICAMP). Instituto de Computação. III. Título.

Informações complementares

Título em outro idioma: Integer linear programming formulations for the min-buying problem

Palavras-chave em inglês:

Pricing

Integer programming

Combinatorial optimization

Área de concentração: Ciência da Computação

Titulação: Mestre em Ciência da Computação

Banca examinadora:

Rafael Crivellari Saliba Schouery [Orientador]

Ruben Interian Kovaliova

Mauro Henrique Mulati

Data de defesa: 26-02-2025

Programa de Pós-Graduação: Ciência da Computação

Objetivos de Desenvolvimento Sustentável (ODS)

Não se aplica

Identificação e informações acadêmicas do(a) aluno(a)

- ORCID do autor: <https://orcid.org/0009-0000-9390-513X>

- Currículo Lattes do autor: <http://lattes.cnpq.br/3650048332187129>

- Prof. Dr. Rafael Crivellari Saliba Schouery
Instituto de Computação - UNICAMP
- Prof. Dr. Mauro Henrique Mulati
Departamento de Informática - UEM
- Prof. Dr. Ruben Interian Kovaliova
Instituto de Computação - UNICAMP

A ata da defesa, assinada pelos membros da Comissão Examinadora, consta no SIGA/Sistema de Fluxo de Dissertação/Tese e na Secretaria do Programa da Unidade.

Agradecimentos

Agradeço a Deus e também a toda a minha família pelo suporte e apoio durante essa caminhada. Em especial, agradeço à minha mãe, Isabel, que foi quem me incentivou a iniciar esse mestrado, e à Micaela, que se tornou minha esposa durante essa jornada, sendo paciente e compassiva nos momentos em que mais precisei, e foi quem me deu dois maravilhosos filhos, os quais me deram a motivação necessária para seguir em frente. Agradeço ao Prof. Dr. Rafael C. S. Schouery pela paciência, ajuda e dedicação para que este trabalho pudesse ser realizado; sem a sua instrução, eu jamais teria conseguido desenvolver esse mestrado. Agradeço a todos os professores do Instituto de Computação que contribuíram grandemente para a minha formação por meio das disciplinas que lá cursei. Agradeço, de forma geral, a todos os funcionários do Instituto de Computação, que se dedicam para torná-lo um excelente ambiente de trabalho, algo que fez muita diferença nas vezes em que estive lá. Por fim, agradeço aos membros da banca examinadora pela disponibilidade para avaliar este trabalho. O presente trabalho foi realizado com apoio do Conselho Nacional de Desenvolvimento Científico e Tecnológico – Brasil (CNPq), processo #131195/2021 – 2.

Resumo

Problemas de precificação são problemas dentro da otimização combinatória que envolvem determinar os preços dos produtos de um vendedor de forma que sua receita, decorrente da venda dos itens, seja máxima. Dentre os problemas de precificação existentes, consideramos o Problema da Compra Mínima, em que um vendedor deseja vender um conjunto I de itens para um conjunto B de compradores. Cada comprador b está disposto a pagar até v_{ib} pelo item i . O vendedor deve então definir preços p_i para cada $i \in I$ de forma a maximizar sua receita, considerando que cada comprador b comprará (se existir) o item mais barato dentre aqueles que $p_i \leq v_{ib}$. Neste trabalho, propomos quatro formulações de Programação Linear para o Problema da Compra Mínima, sendo uma de Programação Linear Inteira (PLI) e três de Programação Linear Inteira Mista (PLIM). Exploramos duas formas principais de se resolver o Problema da Compra Mínima, uma determinando os preços dos itens e os preços pagos pelos compradores, para então indiretamente determinar a alocação dos itens aos compradores, e outra determinando a alocação de forma direta. Foram introduzidas desigualdades válidas em 3 modelos para criar modelos fortalecidos, visando reduzir o tempo de execução e aumentar o número de instâncias solucionadas. Um teorema foi apresentado, demonstrando que é suficiente considerar um subconjunto de preços para cada item, correspondendo às valorações oferecidas a ele. Com isso, foi possível reduzir o número de variáveis e restrições dos modelos. Avaliamos as formulações utilizando três classes de instâncias esparsas, *Characteristics*, *Neighborhood* e *Popularity*, que simulam diferentes cenários práticos de preferências de consumidores, e também instâncias densas, onde todo comprador oferece uma valoração para todo item. Os experimentos mostraram que as desigualdades válidas melhoraram os resultados de 2 modelos. O modelo de PLI fortalecido obteve o melhor resultado para instâncias esparsas, enquanto que um modelo de PLIM obteve o melhor resultado para instâncias densas. Um dos modelos de PLIM obteve o melhor equilíbrio dentre todos os modelos, destacando-se como o melhor modelo de PLIM para instâncias esparsas e resolvendo todas as instâncias densas em um bom tempo.

Abstract

Pricing problems are a class of combinatorial optimization problems that aim to determine the prices of a seller's products to maximize their revenue from sales. Among existing pricing problems, we consider the Min-Buying Problem, where a seller wishes to sell a set I of items to a set B of buyers. Each buyer b is willing to pay up to v_{ib} for item i . The seller must define prices p_i for each $i \in I$ to maximize revenue, assuming that each buyer b will (if possible) purchase the cheapest item for which $p_i \leq v_{ib}$. In this work, we propose four Linear Programming formulations for the Min-Buying Problem: one Integer Linear Programming (ILP) formulation and three Mixed Integer Linear Programming (MILP) formulations. We explore two primary approaches to solving the Min-Buying Problem: one determines the prices of the items and the payments made by buyers, indirectly deriving the allocation of items to buyers, while the other directly determines the allocation. Valid inequalities were introduced in three models to strengthen them, aiming to reduce execution times and solve more instances. A theorem demonstrated that it is sufficient to consider a subset of prices for each item corresponding to the buyers' valuations, reducing the number of variables and constraints. We evaluated the formulations using three classes of sparse instances – *Characteristics*, *Neighborhood*, and *Popularity* –, which simulate different practical consumer preference scenarios, as well as dense or complete instances, where every buyer has a valuation for every item. The experiments showed that valid inequalities improved the results of two models. The strengthened ILP model achieved the best performance for sparse instances, while one MILP model performed best for dense instances. One of the MILP models achieved the best balance overall, excelling as the top MILP model for sparse instances and solving all dense instances efficiently.

Lista de Figuras

1.1	Exemplo de instância para Problemas de Precificação e soluções viáveis para os problemas abordados.	14
5.1	Progresso dos valores de melhor solução e melhor <i>bound</i> para instância da classe <i>Characteristics</i> de tamanho 100, para o modelo (DD-F).	44
5.2	Progresso dos valores de melhor solução e melhor <i>bound</i> para instância da classe <i>Characteristics</i> de tamanho 100, para o modelo (DIV-F).	44

Lista de Tabelas

5.1	Número de instâncias solucionadas e tempo de execução para os modelos (DIB) e (DIB-F).	38
5.2	Gap médio entre valor da relaxação linear e valor da solução ótima para instâncias que foram solucionadas pelos modelos (DIB) e (DIB-F).	39
5.3	Número de instâncias solucionadas e tempo de execução para os modelos (DD) e (DD-F).	40
5.4	Gap médio entre valor da relaxação linear e valor da solução ótima para instâncias que foram solucionadas pelos modelos (DD) e (DD-F).	41
5.5	Média e mediana de nós explorados para instâncias que foram solucionadas pelos modelos (DD) e (DD-F).	41
5.6	Número de instâncias solucionadas e tempo de execução para os modelos (DIV) e (DIV-F).	42
5.7	Gap médio entre valor da relaxação linear e valor da solução ótima para instâncias que foram solucionadas pelos modelos (DIV) e (DIV-F).	42
5.8	Gap médio entre valor da relaxação linear e valor da solução ótima para instâncias que foram solucionadas pelos modelos (DD-F) e (DIV-F).	43
5.9	Média e mediana de nós explorados para instâncias que foram solucionadas pelos modelos (DD-F) e (DIV-F).	45
5.10	Número médio de variáveis inteiras após o <i>presolve</i> para instâncias que foram solucionadas pelos modelos (DD-F) e (DIV-F).	45
5.11	Número de instâncias solucionadas e tempo de execução para os modelos (DIV-F) e (DIB-F).	46
5.12	Número médio de variáveis inteiras e restrições após o <i>presolve</i> para instâncias que foram solucionadas pelos modelos (DIV-F) e (DDI).	47
5.13	Número de instâncias solucionadas e tempo de execução para os modelos (DIV-F) e (DDI).	48
5.14	Gap médio entre valor da relaxação linear e valor da solução ótima para instâncias solucionadas pelos modelos (DIV-F) e (DDI).	48
5.15	Média e mediana de nós explorados para instâncias que foram solucionadas pelos modelo (DIV-F) e (DDI).	49
5.16	Número de instâncias solucionadas e tempo de execução para os modelos (DIB-F) e (DDI).	50
5.17	Gap médio entre valor da relaxação linear e valor da solução ótima para instâncias solucionadas pelos modelos (DIB-F) e (DDI).	51
5.18	Número de instâncias densas solucionadas e tempo de execução para todos os modelos.	51
5.19	Número médio de variáveis inteiras e restrições após o <i>presolve</i> para instâncias densas solucionadas pelos modelos (DIV) e (DDI).	52

5.20	Gap médio entre valor da relaxação linear e valor da solução ótima para instâncias densas solucionadas pelos modelos (DIV) e (DDI).	52
5.21	Tempo médio de execução após <i>presolve</i> e relaxação linear do nó raiz para instâncias densas solucionadas pelos modelos (DIV) e (DDI).	52

Sumário

1	Introdução	12
2	Conceitos Preliminares	16
2.1	Programação Linear	16
2.2	Programação Linear Inteira e Inteira Mista	18
2.2.1	<i>Branch-and-Bound</i>	19
2.3	Algoritmos de Aproximação	20
3	Revisão Bibliográfica	21
3.1	Formulações para Problemas Relacionados	23
4	Formulações para o Problema da Compra Mínima	26
4.1	Formulação de Programação Linear Inteira	26
4.2	Formulação de Programação Linear Inteira Mista	29
4.2.1	Restrições adicionais	31
4.3	Formulação Reduzida de Programação Linear Inteira Mista	32
4.4	Formulação de Programação Linear Inteira Mista Baseada na Alocação . .	34
5	Experimentos e Resultados	36
5.1	Instâncias	36
5.2	Resultados	37
5.2.1	Modelos de Programação Linear Inteira	37
5.2.2	Modelos de Programação Linear Inteira Mista com Decisão Direta .	39
5.2.3	Modelo de Programação Linear Inteira Mista com Decisão Indireta	41
5.2.4	Modelo de Programação Linear Inteira Mista com Decisão Direta e Por Inveja	46
5.3	Instâncias Densas	50
6	Conclusão	53
	Referências Bibliográficas	55

Capítulo 1

Introdução

Compra e venda são relações comerciais presentes no dia a dia da maioria das pessoas. Elas ocorrem de maneira bem simples: um produto é posto à venda por um determinado valor, e uma pessoa disposta a desembolsar esse valor pode pagar para adquiri-lo. No entanto, decidir qual preço colocar em um produto pode se tornar um dilema para os vendedores. Quando um produto é colocado à venda, dois dos principais fatores que irão determinar altas ou baixas vendas são o seu preço e a sua qualidade em relação aos demais concorrentes [27]. Itens caros podem proporcionar alto valor agregado, mas podem gerar baixo volume de vendas; itens baratos podem levar o vendedor ao prejuízo, apesar do maior volume de vendas. Portanto, o preço dos seus produtos é um fator determinante para o sucesso de uma empresa.

Até mesmo uma empresa com produtos diferentes de uma mesma categoria pode enfrentar problemas para determinar seus preços, já que alguns produtos podem ser substitutos parciais de outros [23] – isto é, podem facilmente ser substituídos uns pelos outros, já que possuem uma finalidade parecida –, pois o preço baixo de um produto pode desviar a demanda de outro à medida que os clientes percebem as vantagens de trocar um pelo outro.

Os *problemas de precificação* envolvem determinar os preços dos produtos de um vendedor de maneira que sua receita (soma dos valores de todas as vendas), ao final, seja máxima. Por exemplo, suponha uma empresa de celulares que fabrica alguns modelos distintos. A empresa pode querer descobrir quais são os melhores preços para os seus celulares, de tal forma que seus clientes comprem seus celulares em vez dos celulares das suas concorrentes. O detalhe é que os preços precisam ser competitivos, mas não podem ser muito baixos para que a empresa não tenha prejuízo. Dado seu aspecto prático, os problemas de precificação foram explorados de forma ampla ao longo do tempo, em diferentes situações e com abordagens distintas [23, 27, 24, 1, 16].

Uma das abordagens foi a introduzida por Rusmevichientong et al. [25], chamada *não-paramétrica*. Ela aborda o problema por meio de uma ótica que envolve o conhecimento de poucas informações, como valorações dos compradores aos produtos. Dentro da abordagem não-paramétrica, temos um vendedor que coloca n produtos (os quais chamaremos de *itens*) à venda – representados por um conjunto $I = [n]$ em que $[n]$ denota o conjunto $\{1, \dots, n\}$, e temos m compradores – representados por um conjunto $B = [m]$ – que desejam adquirir um item. O vendedor possui informações sobre as valorações dos

Usa v_{ib} para encontrar π_i

compradores aos itens – representadas por uma matriz v indexada por $I \times B$, onde v_{ib} é o valor que o comprador b dá ao item i , isto é, quanto ele deseja pagar, no máximo, por aquele item – e, a partir delas, atribui preços π_i aos itens para maximizar sua receita.

Os itens podem possuir *oferta ilimitada*, onde não há limitação em relação à sua disponibilidade, ou *oferta limitada*, onde, para cada item, há um número máximo de cópias disponíveis para serem vendidas. Os compradores possuem *demanda unitária*, isto é, desejam adquirir somente um item dentre aqueles que lhes interessam. Além disso, um comprador só pode adquirir um item se este for *viável* para ele, ou seja, se tiver um preço que não ultrapasse o valor da sua valoração ($v_{ib} \geq \pi_i$). Se não existir item viável para um determinado comprador, ele fica sem adquirir itens.

Rusmevichientong et al. [25] introduziu três modelos distintos de problemas de precificação, que possuem como entrada as valorações v , e como saída um vetor de preços $\pi = (\pi_1, \pi_2, \dots, \pi_n)$ e uma matriz binária de alocações a indexada por $I \times B$ – onde a_{ib} indica se o comprador b comprou o item i – que maximizam a receita do vendedor, sendo que as alocações estão sujeitas a certas regras a depender do modelo.

No *Problema da Compra Máxima (Max-Buying Problem)*, que é a base para os demais modelos, cada comprador deve adquirir seu item viável de valor máximo, ou seja, o item mais caro dentre aqueles que lhe são viáveis (caso exista algum).

No *Problema da Compra por Preferência (Rank-Buying Problem)*, além das valorações, há também informações sobre a ordem de preferência dos itens para os compradores, e os compradores devem adquirir seu item de maior preferência dentre os viáveis (caso exista algum). O Problema da Compra por Preferência representa um comportamento comum, onde um consumidor pode preferir comprar um item mais caro dentre as opções disponíveis (que ele consegue pagar), pois o item lhe agrada mais.

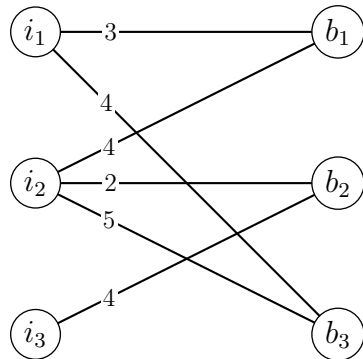
Por fim, no *Problema da Compra Mínima (Min-Buying Problem)* cada comprador deve adquirir seu item viável, caso exista algum, de preço mínimo, ou seja, se um comprador b recebeu um item i , então $\pi_i \leq v_{ib}$ e para todo item j tal que $\pi_j \leq v_{jb}$, temos que $\pi_i \leq \pi_j$. O Problema da Compra Mínima pode representar, muitas vezes, o comportamento de uma significativa parcela dos consumidores. Pensemos, por exemplo, em quantas pessoas conhecemos que, em determinados cenários, escolheriam comprar o item mais barato dentre algumas opções que lhes interessam.

A partir do conceito de *utilidade*, que é definida por $u_b = v_{ib} - \pi_i$ caso o comprador b receba o item i , e $u_b = 0$ caso não receba nenhum item, Guruswami et al. [16] introduziu o *Problema de Precificação Livre de Inveja (Envy-free Pricing Problem)*, onde cada comprador recebe um item viável que maximize sua utilidade, isto é, $u_b \geq 0$ e $u_b \geq v_{jb} - \pi_j$ para todo item $j \in I$.

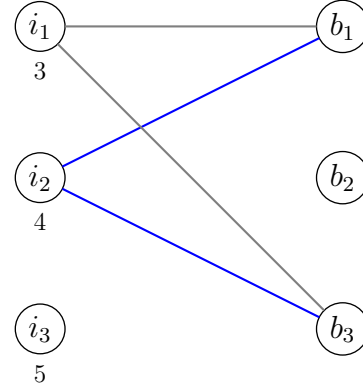
Para cada um dos problemas acima, podemos considerar uma variante com *escada de preços*, que é uma restrição na ordem dos preços dos produtos de tal forma que $\pi_1 \geq \pi_2 \geq \dots \geq \pi_n$, ou seja, para uma precificação ser válida ela deve respeitar a ordem estabelecida. Um exemplo de utilização de uma escada de preços seria uma empresa querer garantir que um produto de maior qualidade sempre tenha um preço maior que um produto de qualidade inferior. Além disso, também podemos considerar um caso particular que são os *orçamentos uniformes*, onde cada comprador $b \in B$ possui um valor V_b e, para cada item $i \in I$, $v_{ib} \in \{0, V_b\}$.

1	0	1
0	1	0
0	0	0

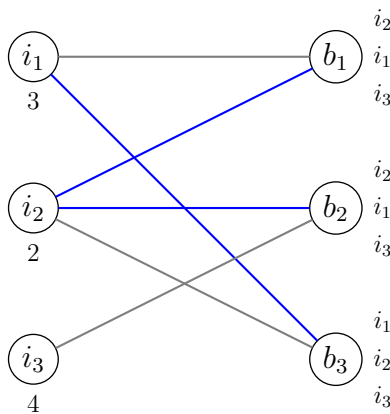
Na Figura 1.1, temos um exemplo de instância para Problemas de Precificação, e também exemplos de soluções viáveis, **mas não necessariamente ótimas**, para cada problema abordado.



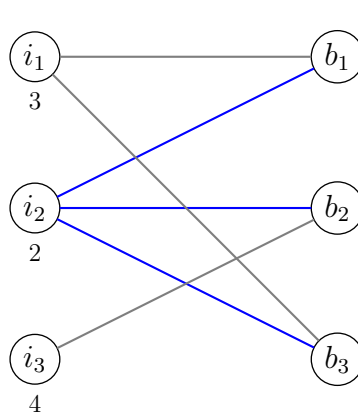
(a) Exemplo de instância.



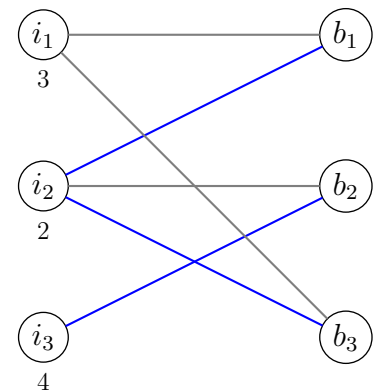
(b) Solução viável para o Problema da Compra Máxima.



(c) Solução viável para o Problema da Compra por Preferência.



(d) Solução viável para o Problema da Compra Mínima.



(e) Solução viável para o Problema de Precificação Livre de Inveja.

Figura 1.1: Exemplo de instância para Problemas de Precificação e soluções viáveis para os problemas abordados. Em (a), temos a instância, onde i_1 , i_2 e i_3 são os itens postos a venda, b_1 , b_2 e b_3 são os compradores, e as arestas entre compradores e itens indicam os valores que os compradores estão dispostos a pagar pelos itens. Em (b), (c), (d) e (e) temos soluções para os problemas da Compra Máxima, Compra por Preferência, Compra Mínima e Precificação Livre de Inveja, respectivamente, onde os preços dos itens estão localizados abaixo dos vértices, as arestas azuis representam a alocação para os preços dados, e as arestas cinzas indicam itens que são viáveis para os compradores. Em (c) temos, para cada comprador, os itens em ordem de preferência ao lado dos vértices correspondentes.

Para o Problema da Compra Máxima, colocando o preço 3 no item i_1 , 4 no item i_2 , e 5 no item i_3 , o vendedor faz com que os compradores b_1 e b_3 comprem o item i_2 , mesmo havendo outro item viável mais barato para eles. Como não há itens viáveis para o comprador b_2 , ele não compra nenhum item.

Para o Problema da Compra por Preferência, assumindo que os compradores b_1 e b_2 preferem os itens i_2 , i_1 e i_3 , nesta ordem, o comprador b_3 prefere os itens i_1 , i_2 e i_3 , e o vendedor coloca preços 3, 2 e 4 nos itens i_1 , i_2 e i_3 , respectivamente, os compradores b_1 e b_2 compram o item i_2 e o comprador b_3 compra o item i_1 , mesmo havendo outro item viável mais barato para b_3 , pois i_1 é o item que ele mais prefere.

Já para o Problema da Compra Mínima, colocando os preços 3, 2 e 4 nos itens i_1 , i_2 e i_3 , respectivamente, o vendedor faz com que todos os compradores comprem o item i_2 , pois é o item viável mais barato para eles. Note que se o vendedor coloca preços 5, 4 e 4 nos itens i_1 , i_2 e i_3 , respectivamente, ele maximiza sua receita, vendendo o item i_2 para os compradores b_1 e b_3 , e o item i_3 para o comprador b_2 , obtendo uma receita igual a 12.

Por fim, para o Problema de Precificação Livre de Inveja, se o vendedor colocar o preço 3 no item i_1 , 2 no item i_2 , e 4 no item i_3 , o comprador b_1 compra o item i_2 , maximizando sua utilidade em relação ao item i_1 . O comprador b_2 compra o item i_3 , mesmo o item i_2 fornecendo a mesma utilidade, que é zero, e aumenta a receita do vendedor por ser mais caro. Por fim, o comprador b_3 compra o item i_2 , que também maximiza sua utilidade.

Diferente de problemas como o Problema da Compra por Preferência e o Problema de Precificação Livre de Inveja, o Problema da Compra Mínima foi muito pouco explorado na literatura, não se conhecendo, até o momento, formulações de Programação Linear para resolvê-lo. Neste trabalho, apresentamos 4 formulações de Programação Linear Inteira para o Problema da Compra Mínima, considerando oferta ilimitada e demanda unitária. No Capítulo 2, definimos formalmente os conceitos relevantes a esta dissertação, como Programação Linear Inteira e Inteira Mista, e então apresentamos, no Capítulo 3, uma revisão bibliográfica sobre os problemas de precificação, abordando, na Seção 3.1, formulações para o Problema da Compra Máxima e para o Problema de Precificação Livre de Inveja. No Capítulo 4, apresentamos as formulações desenvolvidas e o teorema que possibilitou a redução de variáveis e restrições nos modelos. No Capítulo 5, descrevemos nossos resultados, comparando as formulações desenvolvidas, e no Capítulo 6, concluímos a dissertação indicando possíveis abordagens para trabalhos futuros.

Capítulo 2

Conceitos Preliminares

A *otimização combinatória* é um campo da computação e da matemática aplicada que busca solucionar problemas nos quais o conjunto de soluções é finito ou infinitamente enumerável. Nesses problemas, existe, em geral, um número muito grande de soluções viáveis, e enumerá-las pode levar muito tempo. A otimização combinatória estuda formas de projetar algoritmos que encontrem soluções para esses problemas de maneira eficiente, isto é, que executem em tempo polinomial no tamanho da representação da entrada.

Se a solução de um problema consiste em apenas responder “sim” ou “não” para uma determinada entrada, esse problema é chamado de *Problema de Decisão*, e chamamos de P a classe de todos os problemas de decisão que podem ser resolvidos em tempo polinomial na representação da entrada. Chamamos de NP a classe de problemas de decisão onde é possível verificar se a resposta é “sim” em tempo polinomial. Por fim, chamamos de NP-difíceis os problemas, de decisão ou não, que são tão difíceis quanto qualquer problema em NP.

Neste capítulo, serão abordados conceitos necessários para se compreender alguns métodos de resolução de problemas de otimização combinatória, com os quais será possível entender as soluções propostas neste trabalho, passando por programação linear (2.1), *Branch-and-Bound* (2.2.1) e algoritmos de aproximação (2.3).

2.1 Programação Linear

A *Programação Linear* (PL) é uma técnica para modelar problemas que podem ser descritos pela maximização ou minimização de uma função linear, composta pelas variáveis de decisão do problema, chamada de *função objetivo*. As *restrições* são equações ou inequações lineares, também compostas pelas variáveis de decisão, que são utilizadas para impor certos limites aos valores que as variáveis, ou alguma combinação linear delas, podem assumir.

Um programa linear pode ser descrito genericamente como

$$(P) \quad \text{maximize } c^T x \tag{2.1}$$

$$\text{sujeito a } Ax \leq b \tag{2.2}$$

$$x \in \mathbb{R}_+^n \tag{2.3}$$

onde $x \in \mathbb{R}_+^n$ são as variáveis do problema, $c \in \mathbb{R}^n$ são os coeficientes de x na função objetivo, $b \in \mathbb{R}^m$ e $A \in \mathbb{R}^{m \times n}$. Se $x' \in \mathbb{R}_+^n$ satisfaz as restrições, isto é, $Ax' \leq b$, x' é uma *solução viável* para o problema. Seja $\mathbb{X}$ o conjunto de todas as soluções viáveis, denominado *região viável*. Se $x^* \in \mathbb{X}$ é uma solução viável e maximiza a função objetivo, isto é, $c^T x^* \geq c^T x'$ para todo $x' \in \mathbb{X}$, x^* é uma *solução ótima* para o problema. Se $\mathbb{X} = \emptyset$, o problema é dito *inviável*, ou seja, não há solução viável para ele. Se, para todo $M \in \mathbb{R}$, existe $x' \in \mathbb{X}$ tal que $c^T x' \geq M$, então dizemos que (P) é *ilimitado*, ou seja, o valor da sua função objetivo pode crescer arbitrariamente. Caso contrário, (P) é *limitado*.

O programa linear descrito acima é chamado de *forma canônica*. Mas existem outras maneiras de se escrever um programa linear. Podemos minimizar a função objetivo ao invés de maximizar e ter restrições que limitem inferiormente uma combinação linear das variáveis, ou ainda que exijam uma igualdade, por exemplo. No entanto, sempre será possível obter um programa linear equivalente na forma canônica.

As restrições do programa linear delimitam o poliedro $\mathbb{P}(A, b)$, que é definido por $\{x \in \mathbb{R}_+^n \mid Ax \leq b\}$. Note que $\mathbb{X} = \mathbb{P}(A, b)$. Além disso, se $\mathbb{P}(A, b)$ é não-vazio e o programa linear for limitado, existe uma solução ótima x^* correspondente a um vértice de $\mathbb{P}(A, b)$.

Outro tópico muito importante dentro da Programação Linear é o de programas primais e duais. O problema original é chamado de *primal*, geralmente representado por (P), e a partir dele é gerado o problema *dual*, geralmente representado por (D), que consiste em

$$(D) \quad \text{minimize } b^T y \tag{2.4}$$

$$\text{sujeito a } A^T y \geq c \tag{2.5}$$

$$y \in \mathbb{R}_+^m \tag{2.6}$$

sendo y as variáveis duais.

Existem algumas relações importantes entre os programas primal e dual. Primeiro, o sentido da função objetivo do programa dual é oposto ao do primal. Segundo, cada variável do primal está relacionada a uma restrição do dual e cada restrição do primal a uma variável do dual. Por fim, há o Teorema Forte da Dualidade [14] que relaciona os dois programas:

Teorema 1. *Seja (P) um programa linear e (D) seu programa dual:*

- *Se os programas possuem soluções ótimas x^* e y^* , respectivamente, então $c^T x^* = b^T y^*$.*
- *Se (P) é inviável, então (D) é inviável ou ilimitado.*
- *Se (P) é ilimitado, então (D) é inviável.*

Além disso, há o Teorema Fraco da Dualidade [14] que diz que uma solução viável do dual (D) será sempre um limitante superior do primal (P).

Há alguns algoritmos disponíveis para resolução de problemas de PL, como o *algoritmo de pontos interiores* [19], mas o mais utilizado na prática devido aos seus bons resultados é o *Simplex*, desenvolvido por Dantzig [7].

O Simplex é o método mais conhecido e antigo para resolução de problemas de PL, como explicam Korte e Vygen [20]. Embora no pior caso ele possa ter tempo de execução exponencial, já que o número de vértices do poliedro pode crescer exponencialmente com a quantidade de variáveis, na prática, ele resolve a maioria dos problemas de forma eficiente.

A ideia básica do algoritmo consiste em visitar os vértices do poliedro na direção do gradiente da função objetivo. Ele inicia em um dos vértices do poliedro e, iterativamente, move-se para um vértice vizinho que tenha valor melhor ou igual ao atual. A condição de parada é o encontro de um máximo (ou mínimo, se o problema for de minimização) local, isto é, encontrar um vértice sem vizinhos melhores do que ele. Como a região viável é convexa e a função objetivo é linear, esse ótimo local é, na verdade, um ótimo global.

2.2 Programação Linear Inteira e Inteira Mista

No mundo real, há casos onde precisamos restringir mais os valores das variáveis de um problema, e as restrições de um programa linear podem não ser suficientes. A *Programação Linear Inteira* (PLI) permite modelar problemas nos quais suas variáveis precisam assumir valores inteiros, por meio da adição de restrições de integralidade.

Um programa linear inteiro pode ser descrito como

$$(P) \quad \text{maximize } c^T x \quad (2.7)$$

$$\text{sujeito a } Ax \leq b \quad (2.8)$$

$$x \in \mathbb{Z}_+^n \quad (2.9)$$

sendo que x pode ainda pertencer a algum subconjunto de $\mathbb{Z}_+^n$, como é o caso da *Programação Linear Binária* (PIB), onde $x \in \{0, 1\}^n$. Há também os casos nos quais apenas algumas variáveis do problema precisam ser inteiras, e denominamos esses casos de *Programação Linear Inteira Mista*, ou PLIM. No decorrer do texto, tudo o que for descrito sobre a PLI também pode ser aplicado à PLIM, que será omitida para evitar redundância.

Diferentemente da PL, na PLI nem sempre teremos $\mathbb{X} = \mathbb{P}(A, b)$, lembrando que $\mathbb{X}$ é o conjunto de soluções viáveis do problema e $\mathbb{P}(A, b) = \{x \in \mathbb{R}_+^n \mid Ax \leq b\}$. Isto é, nem todo ponto contido no poliedro $\mathbb{P}(A, b)$ é uma solução viável para o problema de PLI. Portanto, uma solução ótima $x^* \in \mathbb{X}$ pode não ser correspondente a um vértice de $\mathbb{P}(A, b)$. Mas, ainda assim, a delimitação do poliedro é importante, pois o vértice que maximiza $c^T x$ fornece um *limitante superior* para o valor da solução ótima de (P). Analogamente, uma solução viável de um problema de PLI, não necessariamente ótima, fornece um *limitante inferior* para o problema.

A *relaxação linear* consiste na remoção das restrições de integralidade, transformando o problema de PLI em um problema de PL. Na relaxação, $\mathbb{P}(A, b)$ é correspondente à região viável do problema. Entretanto, existe um poliedro $\mathbb{P}(A', b')$, que é o *fecho convexo* dos pontos inteiros, onde a resolução do problema de PLI se reduz à resolução da relaxação. Em $\mathbb{P}(A', b')$, todos os vértices são soluções que respeitam as restrições de integralidade e, portanto, viáveis.

Podem existir muitas formulações diferentes para um problema de PLI, e escolher qual

delas será utilizada para resolver o problema é algo essencial. Sejam (PL') e (PL'') as relaxações de duas formulações distintas para um mesmo problema de PLI, com regiões viáveis $\mathbb{P}(A', b')$ e $\mathbb{P}(A'', b'')$ e com valores ótimos z' e z'' , respectivamente. Se $\mathbb{P}(A', b') \subseteq \mathbb{P}(A'', b'')$, então $z' \leq z''$, e a formulação que deu origem a $\mathbb{P}(A', b')$ é dita mais forte que a que deu origem a $\mathbb{P}(A'', b'')$.

É conhecido que encontrar a solução ótima de um programa linear inteiro é um problema NP-difícil [15], mas, apesar de não se ter garantia de ser possível resolver problemas modelados com PLI em tempo polinomial, ela é muito utilizada na prática porque muitos problemas reais de otimização envolvem decisões que devem ser tomadas em termos de valores inteiros, e também porque há métodos que obtêm excelentes resultados em situações reais, como o *Branch-and-Bound*.

2.2.1 *Branch-and-Bound*

O algoritmo de *Branch-and-Bound*, inicialmente proposto por Land e Doing [21], é o método mais utilizado para a resolução de problemas de PLI. A ideia central do algoritmo consiste em utilizar o método de divisão e conquista para decompor a região viável de um problema de PLI em regiões menores, gerando subproblemas mais fáceis de resolver.

O Branch-and-Bound utiliza a combinação de três fases para tentar encontrar uma solução ótima para o problema, sendo elas:

- *Branching* (ou ramificação);
- *Bounding* (ou obtenção de limitantes);
- *Pruning* (ou poda).

Ele começa com uma solução candidata inicial para o problema de otimização, que pode ser indefinida ou pode ser obtida a partir da solução x^* da relaxação linear. Se x^* não é viável para o problema, uma variável $x_i^* \notin \mathbb{Z}_+^n$ é escolhida para se iniciar o processo de *branching*. Essa fase consiste na divisão da região viável, por meio da criação de dois subproblemas, cada um representando um novo ramo na árvore de busca, e com uma das respectivas restrições adicionais:

$$x_i \leq \lfloor x_i^* \rfloor \tag{2.10}$$

$$x_i \geq \lceil x_i^* \rceil \tag{2.11}$$

Dessa forma, a solução x^* será inviável para os subproblemas, que deverão encontrar novas soluções, que poderão fornecer limitantes globais para o problema. Se a solução de um nó não é inteira, isto é, se ela não respeita todas as restrições de integralidade, ela nos fornece um limitante superior para os subproblemas derivados a partir daquele nó. Se é inteira, ela nos fornece um limitante inferior global. Esse processo se repete sucessivamente, e pode ser visualizado por meio de uma árvore, chamada de *árvore de enumeração*, onde a raiz é o problema original, e cada subproblema é um novo nó.

Se todos os nós forem sendo explorados sem um critério de escolha, a quantidade analisada poderá aumentar exponencialmente. A fase de *pruning* serve para evitar que

nós desnecessários sejam explorados, isto é, que sejam desconsiderados para a criação de novos subproblemas. Um nó pode ser *podado* por motivos de: inviabilidade, limitante e otimalidade.

A poda por inviabilidade ocorre quando o subproblema do nó é inviável, isto é, com região viável vazia. Se não há solução fracionária viável, não há solução inteira viável e, portanto, não faz sentido ramificar o nó.

A poda por limitante acontece quando o valor da solução ótima do nó é pior ou igual que o limitante inferior global. Como suas ramificações só terão valores piores ou iguais, o nó não é mais explorado.

Por fim, a poda por otimalidade ocorre quando a solução ótima do nó é inteira para o problema original, pois não há solução inteira melhor a partir da exploração desse mesmo nó. Nesse caso, a solução do nó pode fornecer um limitante inferior global.

A execução do *Branch-and-Bound* termina quando a diferença entre os limitantes superior e inferior globais é menor que uma certa constante pequena ϵ , isto é, quando os limitantes são iguais na prática. Outros critérios de parada também podem ser utilizados, como um determinado número de iterações ou um tempo limite de execução, mas, nesses casos, a solução encontrada pode não ser ótima.

2.3 Algoritmos de Aproximação

Em problemas da classe NP-difícil, como é o caso do Problema da Compra Mínima, não existe algoritmo que consiga encontrar uma solução ótima em tempo polinomial em relação ao tamanho da entrada, a não ser que $P = NP$. No entanto, em determinados casos, é possível encontrar soluções próximas da ótima em tempo polinomial. Mais especificamente, é possível encontrar soluções que garantidamente estão, no máximo, a um fator α de distância da ótima, que é chamado de *razão de aproximação*.

Seja I uma instância qualquer de um problema e $OPT(I)$ o valor da solução ótima. Um algoritmo é uma α -*aproximação* para o problema se, para toda instância I , executa em tempo polinomial no tamanho de I , e encontra uma solução de valor $SOL(I)$ onde $OPT(I) \leq SOL(I) \leq \alpha OPT(I)$ (para o caso de minimização), com $\alpha \geq 1$.

Para alguns problemas de otimização, é possível projetar um *Esquema de Aproximação de Tempo Polinomial* (*Polynomial-Time Approximation Scheme* - PTAS), que consiste em uma família de algoritmos $\{A_\epsilon\}$, onde para um $\epsilon > 0$, A_ϵ é uma $(1 + \epsilon)$ -aproximação (para o caso de minimização) para o problema considerado.

A classe APX é formada pelos problemas da classe NP para os quais existe um algoritmo de aproximação de razão constante. Em particular, um problema APX-difícil é tal que a existência de um PTAS para esse problema implicaria que $P = NP$.

Capítulo 3

Revisão Bibliográfica

Rusmevichientong et al. [25] introduziram a precificação não-paramétrica. Os autores apresentaram um estudo de caso para a precificação de múltiplos itens, objetivando validar a performance da heurística desenvolvida por eles, e para isso utilizaram dados fornecidos pela *Auto Choice Advisor* (ACA) (um website desenvolvido pela *General Motors* (GM) que recomenda veículos a compradores baseando-se em seus requisitos e orçamentos). Apresentaram os problemas da Compra Máxima, Compra Mínima e Compra por Preferência. Provaram que, quando não se exige uma escada de preços, os problemas são NP-difíceis, e demonstraram que, se imposta uma escada de preços, o Problema da Compra Mínima com orçamentos uniformes pode ser resolvido em tempo polinomial.

O Problema da Compra Máxima é a base para os demais problemas de precificação, e teve seu foco na literatura no que se refere a algoritmos de aproximação. Aggarwal et al. [1] estudaram algoritmos para o caso onde uma mesma empresa tem vários produtos concorrentes entre si. Para o Problema da Compra Máxima no caso onde se exige uma escada de preços e a oferta é limitada, apresentaram uma 4-aproximação. Para o caso de oferta ilimitada onde não se exige uma escada de preços, apresentaram uma formulação de PLI e a partir dela uma $e/(e - 1)$ -aproximação juntamente com um limitante inferior de $16/15$. Apresentaram também uma $(\log m)$ -aproximação para os problemas da Compra Máxima, Compra Mínima e Compra por Preferência. Além disso, mostraram uma 2-aproximação para o Problema da Compra Máxima sem escada de preços e com oferta limitada. Provaram que o Problema da Compra Mínima sem escada de preços e com orçamentos uniformes é NP-difícil. Para a Compra por Preferência com escada de preços, apresentaram um PTAS, através de uma redução do problema ao Problema da Compra Máxima com escada de preços e oferta ilimitada.

Guruswani et al. [16] formalizaram a Precificação Livre de Inveja, sugerida inicialmente por Aggarwal et al. [1], dentro do contexto de leilões combinatórios. Demonstraram que o problema é APX-difícil e apresentaram uma $(2 \ln m)$ -aproximação para ele.

Briest e Krysta [2] estudaram os problemas da Compra Máxima, Compra Mínima e Compra por Preferência, provando que os dois últimos não são aproximáveis em $\Omega(\log^\epsilon |m|)$, para $\epsilon > 0$, a não ser que $\text{NP} \subseteq \text{DTIME}(n^{\log \log n})$. Provaram também que o Problema da Compra Máxima e o Problema da Compra por Preferência, ambos com escada de preços, são NP-difíceis, e para o Problema da Compra Máxima sem escada de preços e com oferta limitada apresentaram uma 2-aproximação.

Para a Precificação Livre de Inveja, Chen e Deng [6] demonstraram que, para o caso de demanda unitária onde cada comprador valora positivamente no máximo dois itens e cada item tem apenas uma cópia disponível, o problema pode ser resolvido em tempo polinomial. Já Heilporn et al. [17] apresentaram uma relação entre a Precificação Livre de Inveja e a Precificação de Rede, mostrando como transformar uma instância do segundo problema em uma instância do primeiro.

Shioda, Tunçel, e Myklebust [26] continuaram estudando a Precificação Livre de Inveja, construindo uma formulação de PLIM com cortes válidos para o problema e explorando-a matematicamente. Também expandiram heurísticas para problemas de precificação propostas anteriormente na literatura, utilizando como fonte de dados o setor de turismo. Myklebust, Sharpe e Tunçel [22] estudaram o mesmo problema, apresentando implementações eficientes das heurísticas de Dobson e Kalish [8], além de melhorias nessas heurísticas, e também uma nova formulação de PLIM.

Chalermsook et al. [4] voltaram a estudar o Problema da Compra Mínima, demonstrando que ele não é aproximável em $\log^{1-\epsilon}(n+m)$, para alguma constante ϵ , a não ser que $\text{NP} \subseteq \text{DTIME}(n^{\log^\delta n})$, onde δ é uma constante que depende de ϵ . Posteriormente, Chalermsook et al. [5] apresentaram um limitante inferior de $\Omega(\min(l^{1-\epsilon}, |I|^{1/2-\epsilon}))$, onde l é um limitante superior no número de valorações não nulas de cada comprador, para alguma constante ϵ .

Fernandes et al. [12] abordaram os mesmos problemas que Heilporn et al. [17]. Apresentaram uma redução da Precificação Livre de Inveja à Precificação de Rede utilizando a transformação de instâncias de Heilporn et al. [17], e desenvolveram duas novas formulações para a Precificação Livre de Inveja, comparando-as com outras duas já existentes na literatura. A primeira foi uma formulação para a Precificação de Rede apresentada por Heilporn et al. [18], que foi convertida para a Precificação Livre de Inveja utilizando uma redução apresentada por Heilporn et al. [17]. Já a segunda foi a formulação apresentada por Myklebust, Sharpe e Tunçel [22]. Demonstraram casos onde suas formulações obtiveram melhor desempenho que as já existentes. Além disso, introduziram três modelos para geração de classes de instâncias de demanda unitária esparsas, com interpretações econômicas distintas.

A primeira classe de instância, nomeada pelos autores de *Characteristics*, considera que cada item tem um conjunto de características e que cada comprador deseja algumas dessas características. Se um item possui todas as características que um comprador deseja, o comprador oferece uma valoração ao item. Dessa forma, dois itens que possuem as mesmas características são desejados pelos mesmos compradores, e compradores acabam desejando itens que não diferem muito entre si.

A segunda classe, nomeada *Neighborhood*, considera os itens e os compradores como pontos em um plano, e que os compradores desejam os itens próximos à sua localização no plano. Se um item está a uma distância de no máximo r de um comprador, esse comprador oferece uma valoração ao item, que é proporcional à distância.

Por fim, os autores nomearam a última classe de *Popularity*, que se baseia na ideia de popularidade de um item, que por sua vez é medida pela quantidade de compradores interessados nele. Dessa forma, um item mais popular tem mais chances de ser desejado que um item menos popular. Ao final, o resultado são poucos itens com muitas valorações

e muitos itens com poucas valorações. O modelo constrói as instâncias por meio de um grafo, limitado a e arestas, que representam as valorações.

Os autores também implementaram um gerador para uma classe de instâncias completas, introduzido por Shioda, Tunçel, e Myklebust [26], isto é, instâncias densas que simulam um cenário onde todo comprador oferece uma valoração para todo item.

Fernandes e Schouery [13] voltaram a estudar o Problema da Compra Máxima, mais especificamente para o caso de oferta limitada. Apresentaram uma $e/(e-1)$ -aproximação aleatorizada e mostraram como tornar o algoritmo determinístico, e também apresentaram um PTAS para a variante com escada de preços, melhorando os resultados da literatura em ambos os casos.

Calvete et al. [3] foram os primeiros a explorar formulações para a Compra por Preferência. Apresentaram formulações não-lineares para o problema, e em seguida, mostraram como linearizá-las, além de abordar um algoritmo *branch-and-cut*. Também investigaram pré-processamentos para reduzir o tamanho das instâncias.

Domínguez et al. [9] apresentaram uma nova versão do problema explorado por Calvete et al. [3], considerando oferta limitada dos produtos. Apresentaram uma formulação de PLIM para o problema e também cortes válidos, adicionados ao modelo por meio de um algoritmo de *branch-and-cut*.

Domínguez, Labbé e Marín [10] introduziram a Compra por Preferência com Empates, onde é permitido que haja empates na ordem de preferência dos compradores, e Domínguez, Labbé e Marín [11] introduziram a Compra por Preferência Livre de Inveja. Ambos os trabalhos apresentaram formulações de PLIM para os respectivos problemas.

3.1 Formulações para Problemas Relacionados

Como ponto de partida para o desenvolvimento deste trabalho, utilizamos duas formulações já existentes na literatura para dois problemas diferentes.

A primeira, uma formulação de PLI introduzida por Aggarwal et al. [1] para o Problema da Compra Máxima, que nomeamos (AFMZ) devido às iniciais dos autores (Aggarwal, Feder, Motwani e Zhu). Assuma $P = \{v_{ib} \mid \forall i \in I, \forall b \in B, v_{ib} > 0\}$, e, sem perda de generalidade, note que a escolha dos preços dos itens pode ser restrita aos elementos de P . A formulação possui como variáveis (todas binárias): y_{bp} para todo $b \in B$ e para todo $p \in P$, onde y_{bp} indica se o comprador b compra um item pelo valor p , e x_{ip} para todo $i \in I$ e para todo $p \in P$, onde x_{ip} indica se o preço p foi atribuído ao item i .

A função objetivo é dada pela soma dos valores de todas as vendas e, portanto, maximiza a receita do vendedor. As restrições (3.2) garantem que todo item i tenha um único preço associado a ele. Por sua vez, as restrições (3.3) dizem que se $y_{bp} = 1$, então existe pelo menos um item i com preço p tal que $p \leq v_{ib}$, isto é, i é viável para b . Por fim, as restrições (3.4) garantem que cada comprador b pague no máximo um preço, assegurando a compra de no máximo um item. Para os casos onde um item não recebe preço, isto é, $\sum_{p \in P} x_{ip} = 0$, ele não é vendido, pois ele nunca estará presente no lado direito das restrições (3.3).

$$(AFMZ) \quad \text{maximize} \quad \sum_{b \in B} \sum_{p \in P} p \cdot y_{bp} \quad (3.1)$$

$$\text{sujeito a} \quad \sum_{p \in P} x_{ip} \leq 1 \quad \forall i \in I \quad (3.2)$$

$$y_{bp} \leq \sum_{i \in I: v_{ib} \geq p} x_{ip} \quad \forall b \in B, \forall p \in P \quad (3.3)$$

$$\sum_{p \in P} y_{bp} \leq 1 \quad \forall b \in B \quad (3.4)$$

$$y_{bp} \in \{0, 1\} \quad \forall b \in B, \forall p \in P \quad (3.5)$$

$$x_{ip} \in \{0, 1\} \quad \forall i \in I, \forall p \in P \quad (3.6)$$

A segunda é uma formulação de PLIM introduzida por Heilporn et al. [18] para a Precificação de Rede, e então convertida para a Precificação Livre de Inveja. Foi nomeada por Fernandes et al. [12] (HLMS), também devido às iniciais dos autores (Heilporn, Labbé, Marcotte e Savard). A formulação possui como variáveis x_{ib} para todo $i \in I$ e para todo $b \in B$, onde x_{ib} indica se um item i foi alocado ao comprador b ; π_i para todo $i \in I$ representando uma precificação, sendo que π_i representa o preço do item i ; e $\hat{p}_{ib}$ para todo $i \in I$ e para todo $b \in B$, em que $\hat{p}_{ib}$ indica o preço que o comprador b pagou pelo item i (π_i , caso b tenha recebido i , e 0, caso contrário). Para cada item i , é associada uma constante $R_i \geq \max\{v_{ib} \mid \forall b \in B\}$.

$$(HLMS) \quad \text{maximize} \quad \sum_{b \in B} \sum_{i \in I} \hat{p}_{ib} \quad (3.7)$$

$$\text{sujeito a} \quad \sum_{i \in I} x_{ib} \leq 1 \quad \forall b \in B \quad (3.8)$$

$$\sum_{i \in I} (v_{ib} \cdot x_{ib} - \hat{p}_{ib}) \geq v_{kb} - \pi_k \quad \forall b \in B, \forall k \in I \quad (3.9)$$

$$v_{ib} \cdot x_{ib} - \hat{p}_{ib} \geq 0 \quad \forall b \in B, \forall i \in I \quad (3.10)$$

$$\hat{p}_{ib} \leq \pi_i \quad \forall b \in B, \forall i \in I \quad (3.11)$$

$$\hat{p}_{ib} \geq \pi_i - R_i(1 - x_{ib}) \quad \forall b \in B, \forall i \in I \quad (3.12)$$

$$x_{ib} \in \{0, 1\} \quad \forall b \in B, \forall i \in I \quad (3.13)$$

$$\hat{p}_{ib} \geq 0 \quad \forall b \in B, \forall i \in I \quad (3.14)$$

$$\pi_i \geq 0 \quad \forall i \in I \quad (3.15)$$

A função objetivo é dada pela soma de todos os valores pagos pelos compradores, maximizando, portanto, a receita do vendedor. As restrições (3.8) garantem que cada comprador receba no máximo um único item. Suponha que $x_{ib} = 0$, assim as restrições (3.10), em conjunto com as restrições (3.14), farão $\hat{p}_{ib} = 0$. Ou seja, as restrições (3.10) e (3.14) forçam $\hat{p}_{ib} = 0$ quando o comprador b não recebe o item i . Agora suponha $x_{ib} = 1$, assim as restrições (3.12), em conjunto com as restrições (3.11), farão $\hat{p}_{ib} = \pi_i$. Ou seja, as restrições (3.12) e (3.11) forçam $\hat{p}_{ib} = \pi_i$ quando o comprador b recebe o item i .

As restrições (3.9) garantem a alocação livre de inveja. Sabemos que, quando um comprador b recebe um item i , temos $\hat{p}_{ib} = \pi_i$, enquanto que para os demais itens $j \in I \setminus \{i\}$ temos $\hat{p}_{jb} = 0$. Assim, ficamos com $\sum_{i' \in I} (v_{i'b} \cdot x_{i'b} - \hat{p}_{i'b}) = v_{ib} - \hat{p}_{ib}$. Dessa forma, a restrição (3.9) garante que nenhum item $k \in I$ irá fornecer uma utilidade maior para o comprador b do que a fornecida pelo item i . Se um comprador b não recebe o item i , então na restrição (3.9) temos $\sum_{i \in I} (v_{ib} \cdot x_{ib} - \hat{p}_{ib}) = 0 \geq v_{kb} - \pi_k$ para todo $k \in I$, e, portanto, não há inveja.

Capítulo 4

Formulações para o Problema da Compra Mínima

Este capítulo apresenta as formulações de Programação Linear desenvolvidas para o Problema da Compra Mínima. Em conjunto, apresentamos também um teorema que possibilitou a construção de modelos menores, visando a redução do tempo de execução e o aumento da quantidade de instâncias solucionadas.

4.1 Formulação de Programação Linear Inteira

Tomamos a formulação de PLI para o Problema da Compra Máxima apresentada por Aggarwal et al. [1] como ponto de partida, pois o Problema da Compra Mínima pode ser visto como um problema mais restrito do que o Problema da Compra Máxima. Portanto, com a adição de algumas restrições, é possível que a formulação de Aggarwal et al. [1] consiga resolver o Problema da Compra Mínima.

A ideia da formulação aqui proposta consiste em determinar os preços dos itens e os preços pagos pelos compradores, e, por meio das restrições, estabelecer um vínculo entre esses preços para que cada comprador pague o preço do seu item viável mais barato. Ou seja, a decisão de qual comprador recebe qual item é feita de forma indireta, por isso nomeamos a formulação de Decisão Indireta Binária (DIB), já que as variáveis utilizadas para essa decisão são binárias. Temos as mesmas variáveis da formulação original, (AFMZ) (3.1): y_{bp} , indicando se o comprador b compra um item pelo valor p , e x_{ip} , indicando se o preço p foi atribuído ao item i . Continue assumindo que $P = \{v_{ib} \mid \forall i \in I, \forall b \in B, v_{ib} > 0\}$ é o conjunto de preços que cada item pode receber e cada comprador pode pagar. Logo iremos mostrar como obter conjuntos menores para cada item e para cada comprador.

A função objetivo soma os preços pagos por todos os compradores, maximizando a receita do vendedor. As restrições (4.2), (4.3) e (4.4) são as mesmas da formulação de Aggarwal et al. [1] para o Problema da Compra Máxima, e a adição das restrições (4.5) garante que cada comprador receba seu item viável mais barato, pois quando $y_{bp} = 1$, não podem existir itens viáveis para o comprador com preço $p' < p$. O Lema 1 demonstra que

(DIB) é uma formulação que modela o Problema da Compra Mínima.

$$(DIB) \quad \text{maximize} \quad \sum_{b \in B} \sum_{p \in P} p \cdot y_{bp} \quad (4.1)$$

$$\text{sujeito a} \quad \sum_{p \in P} x_{ip} \leq 1 \quad \forall i \in I \quad (4.2)$$

$$y_{bp} \leq \sum_{i \in I: v_{ib} \geq p} x_{ip} \quad \forall b \in B, \forall p \in P \quad (4.3)$$

$$\sum_{p \in P} y_{bp} \leq 1 \quad \forall b \in B \quad (4.4)$$

$$\sum_{p' \in P: p' < p, p' \leq v_{ib}} x_{ip'} \leq 1 - y_{bp} \quad \forall b \in B, \forall i \in I, \forall p \in P \quad (4.5)$$

$$y_{bp} \in \{0, 1\} \quad \forall b \in B, \forall p \in P \quad (4.6)$$

$$x_{ip} \in \{0, 1\} \quad \forall i \in I, \forall p \in P \quad (4.7)$$

Lema 1. *A formulação (DIB) modela o Problema da Compra Mínima.*

Demonstração. Seja π um vetor de preços e a uma alocação para o Problema da Compra Mínima. Defina a variável $x_{ip'} = 1$ se o preço do item i é p' e $x_{ip'} = 0$ caso contrário, e $y_{bp'} = 1$ se o preço do item que b recebe na alocação a é p' . Primeiramente, vamos mostrar que (x, y) é uma solução viável para (DIB). As restrições (4.2), (4.4), (4.6) e (4.7) são trivialmente satisfeitas. Se $y_{bp'} = 1$ então existe algum item i viável para b com preço p' , e i é o item viável mais barato para b , já que a é uma alocação para o Problema da Compra Mínima. Concluimos que as restrições (4.3) valem, e também as (4.5), pois não existem itens viáveis para b com preço menor que p' , caso contrário a não seria uma alocação para o Problema da Compra Mínima. Portanto, (x, y) é viável para (DIB) e a receita do vendedor é $\sum_{b \in B} \sum_{p \in P} p \cdot y_{bp}$.

Basta então provar que qualquer solução viável (x, y) de (DIB) induz um vetor de preços π e uma alocação a para o Problema da Compra Mínima. Seja $\pi_i = p'$ se $x_{ip'} = 1$, já que as restrições (4.2) garantem que o item i tenha somente um preço atribuído a ele, e $\pi_i = R_i + \epsilon$, $\epsilon > 0$, caso i não tenha recebido preço, onde $R_i = \max\{v_{ib} \mid b \in B\}$. Se existir p' tal que $y_{bp'} = 1$, então existe i tal que $x_{ip'} = 1$ e $p' \leq v_{ib}$. Ou seja, se o comprador b paga o preço p' , então ao menos um item i com preço p' que é viável para b . Escolha um único i e defina $a_{ib} = 1$. Dessa forma, a é uma alocação de demanda unitária. Como as restrições (4.5) garantem que cada comprador pague o preço do seu item viável mais barato, pois quando $y_{bp'} = 1$ não podem existir itens viáveis para ele com preço $p' < p$, a é uma alocação para o Problema da Compra Mínima com receita precisamente $\sum_{b \in B} \sum_{p \in P} p \cdot y_{bp}$. $\square$

Se considerarmos $P = \{v_{ib} \mid \forall i \in I, \forall b \in B, v_{ib} > 0\}$, a cardinalidade do conjunto fica sujeita à quantidade de valores não nulos em v , o que pode comprometer a formulação em relação ao número de variáveis e restrições, já que o número de variáveis x_{ip} e y_{bp} depende de P . Seja $\|v\|_0$ a quantidade de entradas não nulas em v , isto é, a quantidade de valorações diferentes de zero. O número de variáveis e restrições de (DIB) é $\Theta(\|v\|_0 |I| + \|v\|_0 |B|)$ e $\Theta(|I| + \|v\|_0 |B| + \|v\|_0 |I| |B|)$, respectivamente. Buscamos, então, formas de

reduzir a quantidade de preços que os itens podem receber e que os compradores podem pagar. O Teorema 2 auxilia nessa redução.

Teorema 2. *Seja $\epsilon > 0$ e, para todo $i \in I$, seja $R_i = \max\{v_{ib} \mid \forall b \in B\} + \epsilon$. Sem perda de generalidade, se π é um vetor de preços ótimo para o Problema da Compra Mínima, então, para todo item $i \in I$, $\pi_i \in \{v_{ib} \mid \forall b \in B\} \setminus \{0\} \cup \{R_i\}$, isto é, podemos definir $P_i = \{v_{ib} \mid \forall b \in B\} \setminus \{0\} \cup \{R_i\}$. $\square$*

Demonstração. Suponha que $\pi = (\pi_1, \pi_2, \dots, \pi_n)$ seja um vetor de preços ótimo para o Problema da Compra Mínima, e que, para algum i que é vendido, $\pi_i \notin \{v_{ib} \mid \forall b \in B\}$. Seja π'_i o novo preço do item i , onde $\pi'_i = \min\{v_{ib} \mid b \in B \wedge v_{ib} > \pi_i\}$, e mantenha o preço dos outros itens $j \in I \setminus \{i\}$. Considere agora o vetor de preços π após as mudanças, e note que apenas os compradores que adquiriram i podem vir a trocar de item, já que o preço de i aumentou. Porém, o item i não deixou de ser viável para quem o adquiriu. Portanto, os compradores que trocarem de item pagam um preço maior ou igual a π_i , e o lucro do vendedor não diminui. Se pagam um preço igual, o novo vetor de preços também é ótimo. Se pagam um preço maior, isso contradiz a hipótese de que π era um vetor ótimo. Para os casos onde não há troca de item, a receita do vendedor aumenta, pois $\pi'_i > \pi_i$, e isso também contradiz a hipótese inicial. Se existir algum item i vendido com $\pi_i = 0$, podemos definir $\pi_i = \min\{v_{ib} \mid b \in B \wedge v_{ib} > 0\}$, e a receita do vendedor não diminui. Para que um item i não seja vendido, basta definir $\pi_i = R_i$. Logo, para todo item $i \in I$, $\pi_i \in \{v_{ib} \mid b \in B\} \setminus \{0\} \cup \{R_i\}$. $\square$

Portanto, a partir do Teorema 2, podemos dizer que sempre há uma solução ótima onde os preços dos itens vendidos são exatamente algum valor não nulo oferecido para eles.

Para cada comprador $b \in B$, podemos obter um conjunto $P_b = \bigcup_{i \in I \mid v_{ib} > 0} P_i$, que é o conjunto dos preços dos itens que o comprador realmente pode vir a comprar, ou seja, os preços que o comprador b pode pagar. Dessa forma, conseguimos reduzir o número de variáveis para $O(\|v\|_0 + \|v\|_0|B|)$ e o número de restrições para $O(|I| + \|v\|_0|B| + \|v\|_0|I||B|)$, no pior caso, onde todo comprador oferece um valor para todo item.

Apresentamos, ainda, algumas variações para as restrições (4.5):

$$\sum_{\substack{p' \in P_i: \\ p' < p \\ v_{ib} \geq p'}} x_{ip'} \leq 1 - \sum_{\substack{p'' \in P_b: \\ p'' \geq p}} y_{bp''} \quad \forall b \in B, \forall i \in I, \forall p \in P_b \quad (4.8)$$

$$\sum_{\substack{i \in I: \\ v_{ib} > 0}} \sum_{\substack{p' \in P_i: \\ p' < p \\ v_{ib} \geq p'}} x_{ip'} \leq |I|(1 - y_{bp}) \quad \forall b \in B, \forall p \in P_b \quad (4.9)$$

As restrições (4.8) são mais fortes do ponto de vista da relaxação linear do que as restrições (4.5), pois do lado direito da desigualdade temos um somatório envolvendo todas as variáveis $y_{bp''}$, com $p'' \geq p$, dizendo que se o comprador pagou um preço p'' , então o item i não pode ser viável para o comprador e ter preço $p' < p$. Já as restrições

(4.9) são menores em número, mas continuam garantindo a compra mínima, pois quando $y_{bp} = 1$ não podem existir itens viáveis para o comprador com preço $p' < p$. Elas podem ser obtidas pela soma das restrições (4.5) e são mais fracas.

Temos a mesma quantidade de restrições (4.8) que restrições (4.5), e temos $O(\|v\|_0|B|)$ restrições (4.9). Dessa forma, a quantidade de restrições da formulação se mantém a mesma caso as restrições (4.5) sejam substituídas pelas (4.8), e cai para $O(|I| + \|v\|_0|B|)$ caso sejam substituídas pelas restrições (4.9).

Além disso, também podemos demonstrar que as restrições (4.4) são desnecessárias para a formulação. Quando temos $y_{bp} = 1$, sabemos que existe algum $x_{ip} = 1$, devido às restrições (4.3). As restrições de compra mínima, (4.5), (4.8) ou (4.9), garantem que $y_{bp'} = 0$ para todo $p' < p$, pois se existisse $y_{bp'} = 1$ para algum $p' > p$, y_{bp} deveria ser igual a zero por conta das mesmas restrições. Portanto, as restrições (4.4) não são necessárias para que os compradores paguem somente um preço. No entanto, na prática, essas restrições são cortes válidos e ajudam no desempenho da formulação.

4.2 Formulação de Programação Linear Inteira Mista

A formulação de PLIM para a Precificação Livre de Inveja apresentada por Heilporn et al. [18] e nomeada por Fernandes et al. [12] de (HLMS) (3.8) é a inspiração para o desenvolvimento desta formulação. A ideia consiste em determinar os preços dos itens e, a partir disso, saber quais são viáveis para cada um dos compradores. Em seguida, determinar as alocações de compra mínima, inferindo os preços pagos ao vendedor por cada um dos compradores. Ou seja, nessa formulação, a decisão de qual item cada comprador recebe é feita de forma direta, por isso a nomeamos de Decisão Direta (DD). Para isso, temos as mesmas variáveis da formulação original (x_{ib} indicando se o comprador b recebeu ou não o item i , π_i representando o preço do item i , e $\hat{p}_{ib}$, indicando o preço que o comprador b pagou pelo item i), apresentada na Seção 3.1, com a adição das variáveis binárias y_{ib} para todo $i \in I$ e para todo $b \in B$, em que $y_{ib} = 1$ indica que o item i é viável para o comprador b considerando o preço π_i , e $y_{ib} = 0$ indica que não é viável.

Ainda, associamos a cada comprador b um conjunto de itens $S_b = \{i \in I \mid v_{ib} > 0\}$, que são os itens que o comprador deseja adquirir (lembre-se, adquirir somente um deles). Também temos os parâmetros adicionais $R_b = \max\{v_{ib} \mid \forall i \in S_b\}$, representando o maior valor que o comprador b pode pagar, e $\epsilon = \min\{v_{ib} - v_{i'b'} \mid \forall i, i' \in I, \forall b, b' \in B, v_{ib} \neq v_{i'b'}\}$, que é a menor diferença positiva de preço dentre todas as valorações oferecidas.

As restrições (3.10) da formulação (HLMS) (3.8) são removidas, já que são referentes à Precificação Livre de Inveja. São adicionadas as restrições (4.15) e (4.16), que garantem que cada comprador receba seu item viável de valor mínimo. Do Lema 2, segue que (DD) é uma formulação que modela o Problema da Compra Mínima.

Como nenhuma variável de (DD) é indexada pelo preço, diferentemente do que acontece em (DIB), o tamanho da formulação é menor. Temos $\Theta(\|v\|_0 + |I|)$ variáveis e

$\Theta(\|v\|_0 + |B|)$ restrições, fazendo com que a formulação seja mais leve.

$$(DD) \quad \text{maximize} \quad \sum_{b \in B} \sum_{i \in S_b} \hat{p}_{ib} \quad (4.10)$$

$$\text{sujeito a} \quad \sum_{i \in S_b} x_{ib} \leq 1 \quad \forall b \in B \quad (4.11)$$

$$v_{ib} \cdot x_{ib} - \hat{p}_{ib} \geq 0 \quad \forall b \in B, \forall i \in S_b \quad (4.12)$$

$$\hat{p}_{ib} \leq \pi_i \quad \forall b \in B, \forall i \in S_b \quad (4.13)$$

$$\hat{p}_{ib} \geq \pi_i - R_i \cdot (1 - x_{ib}) \quad \forall b \in B, \forall i \in S_b \quad (4.14)$$

$$y_{ib} \geq \frac{v_{ib} - \pi_i + \epsilon}{v_{ib} + \epsilon} \quad \forall b \in B, \forall i \in S_b \quad (4.15)$$

$$\pi_i \geq \sum_{j \in S_b} \hat{p}_{jb} - R_b(1 - y_{ib}) \quad \forall b \in B, \forall i \in S_b \quad (4.16)$$

$$\hat{p}_{ib} \geq 0 \quad \forall b \in B, \forall i \in S_b \quad (4.17)$$

$$\pi_i \geq 0 \quad \forall i \in I \quad (4.18)$$

$$x_{ib} \in \{0, 1\} \quad \forall b \in B, \forall i \in S_b \quad (4.19)$$

$$y_{ib} \in \{0, 1\} \quad \forall b \in B, \forall i \in S_b \quad (4.20)$$

Lema 2. *A formulação (DD) modela o Problema da Compra Mínima.*

Demonstração. Seja π um vetor de preços e x uma alocação para o Problema da Compra Mínima. Defina a variável $\hat{p}_{ib} = \pi_i$ se i está alocado a b e $\hat{p}_{ib} = 0$ caso contrário, e $y_{ib} = 1$, se $v_{ib} \geq \pi_i$, e 0, caso contrário. Primeiramente, vamos mostrar que $(x, \pi, \hat{p}, y)$ é uma solução viável para (DD). As restrições (4.11), (4.13), (4.15), (4.17), (4.18), (4.19) e (4.20) são trivialmente satisfeitas. Quando $x_{ib} = 1$, sabemos que i é viável para b e $\hat{p}_{ib} = \pi_i$. Portanto, $v_{ib} \geq \hat{p}_{ib}$ e as restrições (4.12) valem. Já quando $x_{ib} = 0$, i não está alocado para b e $\hat{p}_{ib} = 0$. Portanto, as restrições (4.12) continuam valendo. Além disso, quando $x_{ib} = 1$, as restrições (4.14) se reduzem a $\hat{p}_{ib} \geq \pi_i$ e são válidas, e quando $x_{ib} = 0$ se reduzem a $\hat{p}_{ib} \geq \pi_i - R_i$ e continuam valendo, já que $\pi_i - R_i \leq 0$. Agora, note que quando $y_{ib} = 1$, as restrições (4.16) se reduzem a $\pi_i \geq \sum_{j \in S_b} \hat{p}_{jb}$. Como $\sum_{j \in S_b} \hat{p}_{jb}$ representa o preço pago pelo comprador b , e b recebeu seu item viável mais barato, todo item i viável para b tem preço maior ou igual ao que b efetivamente pagou. Concluimos que as restrições (4.16) valem. Agora, quando $y_{ib} = 0$, as restrições (4.16) se reduzem a $\pi_i \geq \sum_{j \in S_b} \hat{p}_{jb} - R_b$ e continuam valendo, já que $\sum_{j \in S_b} \hat{p}_{jb} - R_b \leq 0$. Concluimos que $(x, \pi, \hat{p}, y)$ é uma solução viável para (DD) e $\sum_{b \in B} \sum_{i \in S_b} \hat{p}_{ib}$ é exatamente a receita do vendedor.

Basta então provar que qualquer solução viável $(x, \pi, \hat{p}, y)$ de (DD) é tal que x é uma alocação para o Problema da Compra Mínima considerando π . Por meio de (4.11), concluimos que x é uma alocação de demanda unitária. Por meio de (4.12) e (4.17), sabemos que $\hat{p}_{ib} = 0$ quando b não compra i ($x_{ib} = 0$), e por meio de (4.13) e (4.14), sabemos que $\hat{p}_{ib} = \pi_i$ quando b compra i ($x_{ib} = 1$). Como x é uma alocação de demanda unitária, $\sum_{i \in S_b} \hat{p}_{ib} = \hat{p}_{jb} = \pi_j$, onde $x_{jb} = 1$, representa exatamente o preço pago por b pelo item que ele comprou, caso tenha comprado algum.

Quando o item i é viável para o comprador b , temos nas restrições (4.15)

$$y_{ib} \geq \frac{v_{ib} - \pi_i + \epsilon}{v_{ib} + \epsilon} \geq \frac{0 + \epsilon}{v_{ib} + \epsilon} > 0,$$

sendo que $\frac{\epsilon}{v_{ib} + \epsilon} < 1$, fazendo com que $y_{ib} = 1$. A partir disso, temos nas restrições (4.16),

$$\pi_i \geq \sum_{j \in I} \hat{p}_{jb} - R_i(1 - y_{ib}) = \sum_{j \in I} \hat{p}_{jb},$$

ou seja, o preço pago por b pelo item que ele adquiriu, caso tenha adquirido algum, deve ser menor ou igual ao preço de qualquer outro item viável para ele. Isto é, x é uma alocação para o Problema da Compra Mínima considerando π , com $\sum_{b \in B} \sum_{i \in S_b} \hat{p}_{ib}$ sendo exatamente a receita do vendedor. $\square$

Note que, quando o item i não é viável para o comprador b , y_{ib} ficará livre para assumir o valor 0 ou 1, pois, pelo Teorema 2, sabemos que a diferença entre o preço de um item (π_i) e o valor de uma valoração (v_{ib}), sem perda de generalidade, é de pelo menos ϵ em uma solução ótima. Portanto, quando o item não é viável para o comprador, $v_{ib} - \pi_i + \epsilon \leq 0$, implicando em valores menores ou iguais a zero no lado direito das restrições (4.15), e deixando y_{ib} livre para assumir 0 ou 1. Se y_{ib} assumir o valor 1, nas restrições (4.16), o preço de um item i não viável para b ficará limitado inferiormente pelo preço de algum item comprado por b , ou zero caso b não tenha adquirido item algum. Se isso não for um problema, podemos manter y_{ib} em 1 e a solução ótima será encontrada. Já se for um problema, podemos trocar y_{ib} para 0 e deixar o preço do item i livre. Ou seja, os preços dos itens não viáveis não influenciam a solução.

4.2.1 Restrições adicionais

Com a finalidade de fortalecer a formulação (DD) (4.11), fornecemos duas formas de gerar desigualdades válidas. A primeira envolve as restrições (4.21) e (4.22), que têm por objetivo inferir a viabilidade de um item a um comprador por meio da viabilidade desse item a outros compradores.

Seja T_{ib} , para todo $i \in I$ e para todo $b \in B$, o conjunto dos compradores que valoram i em, no máximo, v_{ib} , ou seja, $T_{ib} = \{b' \mid v_{ib'} \leq v_{ib}, \forall b' \in B \setminus \{b\}\}$. As restrições (4.21) fazem com que $y_{ib} = 1$ se o item i for viável para algum comprador $b' \in T_{ib}$. Ou seja, se i é viável para algum comprador $b' \in T_{ib}$, i com certeza também é viável para b . Já as restrições (4.22) fazem com que $y_{ib} = 0$ se o item i não for viável para nenhum comprador b' com $v_{ib'} \geq v_{ib}$. Isto é, se i não é viável para nenhum comprador que oferece a ele uma valoração maior que v_{ib} , então não há como i ser viável para b .

$$y_{ib} \geq \frac{\sum_{b' \in T_{ib}} y_{ib'}}{|T_{ib}|} \quad \forall b \in B, \forall i \in S_b \quad (4.21)$$

$$y_{ib} \leq \sum_{b' \neq b, v_{ib'} \geq v_{ib}} y_{ib'} \quad \forall b \in B, \forall i \in S_b \quad (4.22)$$

Temos $\Theta(\|v\|_0)$ restrições (4.21) e (4.22), e, dessa forma, a formulação ainda continua com $\Theta(\|v\|_0 + |B|)$ restrições.

Seja $\hat{P}_i = (\sigma_1, \sigma_2, \dots, \sigma_m)$, onde $\sigma_j \in B$, $v_{i\sigma_j} > 0$ e $v_{i\sigma_j} \leq v_{i\sigma_{j+1}}$, para todo $j \in \{1, \dots, m-1\}$. As restrições (4.23) são uma segunda forma de gerar cortes válidos, também com o objetivo de realizar inferências sobre as viabilidades dos itens aos compradores. Elas contemplam todos os casos onde, se um item i é viável para um comprador b , ele também deverá ser viável para os compradores b' com $v_{ib'} \geq v_{ib}$, e se ele não for viável para um comprador b , não deverá ser viável para os compradores b' com $v_{ib'} \leq v_{ib}$. Por exemplo, se os compradores b , b' e b'' valoram o item i em 5, 3 e 2, respectivamente, e o i é viável para b'' , podemos afirmar que o i também é viável para b e b' , isto é, $y_{ib''} = 1 \leq y_{ib'} \leq y_{ib}$.

$$y_{i\sigma_j} \leq y_{i\sigma_{j+1}} \quad \forall i \in I, \forall \sigma_j \in \hat{P}_i \quad (4.23)$$

Essas restrições são em quantidade $\Theta(\|v\|_0)$, e, no pior caso, teríamos $\Theta(|I||B|)$ restrições a mais, considerando instâncias densas. Ainda assim, a formulação continua tendo $\Theta(\|v\|_0 + |B|)$ restrições.

4.3 Formulação Reduzida de Programação Linear Inteira Mista

Tomando como ponto de partida a formulação (DD), apresentada na seção anterior, construímos uma formulação mais enxuta, que com a remoção das variáveis x (que representavam a decisão direta de qual comprador compra qual item) leva a uma ideia mais parecida com a formulação (DIB): determinar os preços dos itens e a viabilidade deles para os compradores, e então inferir os preços de compra mínima para cada comprador. A decisão de qual comprador compra qual item é, portanto, indireta, e, por isso, nomeamos a formulação de Decisão Indireta por Viabilidade (DIV), pois o preço pago pelos compradores é inferido a partir da viabilidade dos itens. A formulação conta com as variáveis π_i , representando o preço do item i , y_{ib} , indicando se o item i é ou não viável para o comprador b , e $\hat{p}_b$, que representa o preço pago pelo comprador b .

$$(DIV) \quad \text{maximize} \quad \sum_{b \in B} \hat{p}_b \quad (4.24)$$

$$\text{sujeito a} \quad y_{ib} \geq \frac{v_{ib} - \pi_i + \epsilon}{v_{ib} + \epsilon} \quad \forall b \in B, \forall i \in S_b \quad (4.25)$$

$$\pi_i - v_{ib} \leq (R_i - v_{ib}) \cdot (1 - y_{ib}) \quad \forall b \in B, \forall i \in S_b \quad (4.26)$$

$$\hat{p}_b \leq \pi_i + R_b(1 - y_{ib}) \quad \forall b \in B, \forall i \in S_b \quad (4.27)$$

$$\hat{p}_b \leq \sum_{i \in S_b} (v_{ib} \cdot y_{ib}) \quad \forall b \in B \quad (4.28)$$

$$\pi_i \geq 0 \quad \forall i \in I \quad (4.29)$$

$$\hat{p}_b \geq 0 \quad \forall b \in B \quad (4.30)$$

$$y_{ib} \in \{0, 1\} \quad \forall b \in B, \forall i \in S_b \quad (4.31)$$

A função objetivo soma todos os preços pagos pelos compradores, logo, maximiza a receita do vendedor. As restrições (4.25) fazem com que y_{ib} assumo o valor correspondente à viabilidade do item ao comprador. As restrições (4.26) fazem com que, caso $y_{ib} = 1$, então $\pi_i \leq v_{ib}$, garantindo que π_i seja sempre igual ao valor da valoração mais barata dentre as valorações dos compradores para os quais o item i é viável. As restrições (4.27) garantem que, caso $y_{ib} = 1$, então $\hat{p}_b \leq \pi_i$, ou seja, $\hat{p}_b$ é sempre igual ao preço do item viável mais barato para o comprador. As restrições (4.28) asseguram que o comprador não pague nenhum preço quando não há itens viáveis para ele. Do Lema 3, segue que (DIV) é uma formulação que modela o Problema da Compra Mínima.

Como as variáveis y_{ib} possuem o mesmo significado e são determinadas da mesma forma tanto na formulação (DD) quanto na (DIV), podemos adicionar aqui também as restrições (4.21) e (4.22), ou as restrições (4.23). Como (DIV) não possui as variáveis x_{ib} e também possui menos variáveis indexadas por I , é ainda mais leve que (DD). Temos $\Theta(\|v\|_0 + |I|)$ variáveis e $\Theta(\|v\|_0 + |B|)$ restrições.

Lema 3. *A formulação (DIV) modela o Problema da Compra Mínima.*

Demonstração. Seja π um vetor de preços e x uma alocação para o Problema da Compra Mínima. Defina a variável $\hat{p}_b = \pi_i$, se i está alocado a b , e $\hat{p}_b = 0$, caso nenhum item esteja alocado a ele. Defina também $y_{ib} = 1$, se $v_{ib} \leq \pi_i$, e $y_{ib} = 0$, caso contrário. Primeiramente, vamos mostrar que $(\pi, \hat{p}, y)$ é uma solução viável para (DIV). As restrições (4.25), (4.29), (4.30) e (4.31) são trivialmente satisfeitas. Quando $y_{ib} = 1$, as restrições (4.26) e (4.27) são satisfeitas, pois se i é viável para b sabemos que $\pi_i \leq v_{ib}$, e o preço pago por b deve ser menor ou igual ao preço de i . Quando $y_{ib} = 0$, as restrições (4.26) e (4.27) continuam sendo satisfeitas, pois $\pi_i < R_i$ e $\hat{p}_b < R_b$. Em relação às restrições (4.28), quando $y_{ib} = 1$ temos um limitante superior para $\hat{p}_b$ de pelo menos v_{ib} , e as restrições valem, pois se i é viável para b então $\hat{p}_b \leq \pi_i$, e quando nenhum item é viável para b temos $\hat{p}_b = 0$, e as restrições valem trivialmente. Além disso, $\sum_{b \in B} \hat{p}_b$ é a receita do vendedor.

Vamos mostrar agora que qualquer solução $(\pi, \hat{p}, y)$ viável de (DIV) induz uma alocação x para o Problema da Compra Mínima considerando π . Defina $x_{ib} = 1$ se $y_{ib} = 1$ e $\hat{p}_b = \pi_i$. Caso exista mais de um item que satisfaça essas condições, escolha um único i arbitrariamente dentre esses itens e defina $x_{ib} = 1$. Dessa forma, x é uma alocação de demanda unitária. As restrições (4.26) fazem com que, caso $y_{ib} = 1$, então $\pi_i \leq v_{ib}$. As restrições (4.27) garantem que, caso $y_{ib} = 1$, então $\hat{p}_b \leq \pi_i$. Isso faz com que $\hat{p}_b$ seja sempre igual ao preço do item viável mais barato para o comprador. As restrições (4.28) servem para o caso onde um comprador não possui itens viáveis, assegurando que ele não pague nenhum preço nessa ocasião. Assim x é uma alocação para o Problema da Compra Mínima considerando π , e a receita do vendedor é exatamente $\sum_{b \in B} \hat{p}_b$. $\square$

Como as variáveis y_{ib} possuem o mesmo significado e são determinadas da mesma forma tanto na formulação (DD) quanto na (DIV), podemos adicionar aqui também as restrições (4.21) e (4.22), ou as restrições (4.23).

4.4 Formulação de Programação Linear Inteira Mista Baseada na Alocação

Abandonando o conceito de viabilidade dentro das formulações de Programação Linear Inteira Mista, exploramos aqui uma formulação centrada na alocação, onde, por meio da informação de qual item um comprador adquiriu, obtemos os preços de compra mínima para os outros compradores. Nomeamos a formulação de (DDI), de Decisão Direta e por Inveja, já que um comprador olha para as alocações dos outros compradores para decidir quanto vai pagar, invejando quem está pagando menos do que ele. Introduzimos para cada item i o conjunto $S_i = \{b \mid \forall b \in B, v_{ib} > 0\}$, que contém os compradores que possuem interesse em adquirir o item. Temos como variáveis $\hat{p}_b$, também utilizadas na formulação (DIV), indicando o preço pago por cada comprador, e x_{ib} , que indica se um comprador b adquiriu o item i .

$$\begin{aligned} \text{(DDI)} \quad & \text{maximize } \sum_{b \in B} \hat{p}_b \\ & \text{sujeito a } \sum_{i \in S_b} x_{ib} \leq 1 \quad \forall b \in B \end{aligned} \quad (4.32)$$

$$\hat{p}_b \leq v_{ib'} + R_b \cdot (1 - x_{ib'}) \quad \forall b \in B, \forall i \in S_b, \forall b' \in S_i \wedge v_{ib'} \leq v_{ib} \quad (4.33)$$

$$\hat{p}_b \leq \sum_{i \in S_b} (v_{ib} \cdot x_{ib}) \quad \forall b \in B \quad (4.34)$$

$$\hat{p}_b \geq 0 \quad \forall b \in B \quad (4.35)$$

$$x_{ib} \in \{0, 1\} \quad \forall b \in B, \forall i \in S_b \quad (4.36)$$

Assim como em (DIV), a função objetivo soma todos os preços pagos pelos compradores, maximizando a receita do vendedor. As restrições (4.32) garantem que cada comprador receba somente um item. As restrições (4.33) garantem que o comprador b pague o menor preço dentre os itens que ele deseja adquirir e que foram alocados a alguém que os valora, no máximo, igual a ele. Por fim, as restrições (4.34) servem para o caso em que um comprador não possui itens viáveis, assegurando que ele não pague nenhum preço nessa ocasião. A formulação tem $\Theta(\|v\|_0 + |B|)$ variáveis e $O(|B| + \|v\|_0 |B|)$ restrições. No Lema 4, demonstramos que (DDI) é uma formulação que modela o Problema da Compra Mínima.

Lema 4. *A formulação (DDI) modela o Problema da Compra Mínima.*

Demonstração. Seja π um vetor de preços e x uma alocação para o Problema da Compra Mínima. Seja a variável $\hat{p}_b = \pi_i$ se i está alocado a b , ou igual a zero caso nenhum item tenha sido alocado a ele. Vamos mostrar que $(x, \hat{p})$ é uma solução viável para (DDI). As restrições (4.32), (4.35) e (4.36) são trivialmente satisfeitas. Quando um comprador recebe um item, sabemos que $\pi_i \leq v_{ib}$, já que o item tem que ser viável para o comprador. Nas restrições (4.33), quando $x_{ib'} = 1$ temos $\pi_i \leq v_{ib'}$. Se b também recebe i , as restrições são satisfeitas, pois $\hat{p}_b = \pi_i \leq v_{ib'}$. Se b não recebe i , as restrições também são satisfeitas, pois, como x é uma alocação para o Problema da Compra Mínima considerando π , ele recebe um item j mais barato que i ($\hat{p}_b = \pi_j \leq \pi_i$). Quando $x_{ib'} = 0$ temos $\hat{p}_b \leq v_{ib'} + R_b$

e as restrições valem para os casos onde b recebe ou não i , já que $\hat{p}_b$ sempre será menor que R_b . Já nas restrições (4.34), quando $x_{ib} = 1$ temos um limitante superior para $\hat{p}_b$ e as restrições valem, já que $\hat{p}_b = \pi_i \leq v_{ib}$, e quando b não recebe nenhum item, as restrições também valem, pois, nesse caso, $\hat{p}_b = 0$. Por fim, podemos concluir que a receita do vendedor é $\sum_{b \in B} \hat{p}_b$.

Basta demonstrar, então, que qualquer solução viável $(x, \hat{p})$ de (DDI) induz um vetor de preços π tal que x é uma alocação para o Problema da Compra Mínima. As restrições (4.32) garantem que x é uma alocação de demanda unitária. Seja $\pi_i = \hat{p}_b$ caso $x_{ib} = 1$. Por meio do Teorema 2, sabemos que o preço de um item será igual ao valor de alguma valoração dada a ele. Dessa forma, se o comprador b deseja adquirir o item i e este item foi alocado a um comprador b' com $v_{ib'} \leq v_{ib}$, as restrições (4.33) garantem que b pagará no máximo o valor $v_{ib'}$ ao vendedor. Se olharmos para todos os itens que b deseja e aplicarmos essa mesma regra, iremos garantir que b pague o menor preço possível. Além disso, as restrições (4.34) garantem que b pague zero caso não tenha recebido nenhum item. Ou seja, x é uma alocação para o Problema da Compra Mínima considerando π , com receita do vendedor igual a $\sum_{b \in B} \hat{p}_b$. $\square$

Capítulo 5

Experimentos e Resultados

Neste capítulo, apresentamos os experimentos computacionais realizados para a comparação das formulações apresentadas no capítulo anterior, bem como as discussões decorrentes dos resultados desses experimentos.

Todos os modelos testados foram implementados na linguagem de programação C++, utilizando a API do *software Gurobi* na versão 9,5.1, e compilados com o compilador GNU G++ (GCC) na versão 11,4.0, sobre o sistema Ubuntu 22,04,3 LTS (64-bit). O *software* foi configurado para utilizar somente uma *thread*, a fim de fornecer uma comparação mais justa entre os modelos. Os testes foram executados em uma máquina com processador Intel(R) Xeon(R) E5-2630 2,20GHz, com 20 núcleos físicos de duas *threads* cada e 64GB de memória RAM.

Na seção 5.1, explicamos sobre a seleção das instâncias utilizadas nos testes. Na seção 5.2, abordamos os resultados considerando as instâncias esparsas, das classes *Characteristics*, *Neighborhood* e *Popularity*, e, separadamente na seção 5.3, os resultados para as instâncias densas, que são densas.

5.1 Instâncias

As instâncias utilizadas nos experimentos foram baseadas nos modelos introduzidos por Fernandes et al. [12], e geradas pelo gerador *Unit-Demand Market Models*¹, que implementou os modelos descritos pelos autores.

As instâncias foram geradas de acordo com os parâmetros definidos por Fernandes et al. [12]. Todos os modelos possuem como parâmetro n , representando a quantidade de itens e compradores (quando forem iguais), i representando a quantidade de itens e b representando a quantidade de compradores (quando forem diferentes). Além desses, cada modelo possui seus parâmetros específicos.

Para as instâncias da classe *Characteristics*, representadas pela inicial c , os parâmetros específicos são: c (número de características de cada item), o (número de opções para cada característica), p (quantidade de opções desejadas por um comprador para qualquer característica), l e h (valores mínimo e máximo para as valorações), e d (porcentagem de desvio padrão). Para as instâncias da classe *Neighborhood*, representadas pela inicial n ,

¹<https://github.com/schouery/unit-demand-market-models>

temos como parâmetros específicos: h (multiplicador máximo para definição da valoração de um comprador), r (raio da vizinhança de um comprador), e d (grau esperado). Por fim, para as instâncias da classe *Popularity*, representadas pela inicial p , os parâmetros específicos são: e (número de arestas no grafo), q (valor máximo de qualidade para um item), e d (porcentagem de desvio padrão). Para as instâncias densas, o prefixo utilizado é rc , do inglês *Random Complete*.

Os autores do gerador fornecem uma série de instâncias prontas, disponíveis no repositório. Para identificação dessas instâncias, basta saber o prefixo de cada modelo (c , n , p ou rc), o valor do parâmetro n e o sufixo, que diferencia uma instância dentro de um conjunto com outras geradas com os mesmos parâmetros. Por exemplo, a instância $c-n=500-c=31-p=7-o=8-l=1-h=100-d=0.25.5$ é uma instância da classe *Characteristics*, por conta do prefixo c , com 500 itens e 500 compradores, indicados pelo parâmetro n , e é também a instância de número 5 dentro do conjunto.

Das instâncias disponíveis, selecionamos algumas para compor o conjunto de teste. Para as classes *Characteristics*, *Neighborhood* e *Popularity*, selecionamos 10 instâncias, com sufixos de 0 a 9, com as seguintes quantidades (iguais) de compradores e itens: 100, 150, 300, 500, 1000, 1500 e 2000. Foram selecionadas também 10 instâncias densas para 100, 150 e 200 compradores e itens. As instâncias foram executadas uma vez para cada modelo testado, com tempo limite de 60 minutos.

5.2 Resultados

Testamos os modelos apresentados nas seções anteriores e algumas variações deles, com a finalidade de verificar se o fortalecimento desses modelos levaria a melhores resultados. Testamos o modelo (DIB) e uma variação com as restrições (4.8), que nomeamos (DIB-F), já que as restrições (4.8) são mais fortes que as restrições (4.6); o modelo (DD) e uma variação com a adição das restrições (4.23), que nomeamos (DD-F); o modelo (DIV) e também sua versão com a adição das restrições (4.23), que nomeamos (DIV-F); e o modelo (DDI).

5.2.1 Modelos de Programação Linear Inteira

Os modelos (DIB) e (DIB-F) conseguiram solucionar todas as instâncias da classe *Characteristics* e *Neighborhood* em tempos abaixo de 2 minutos, e a maior parte delas abaixo de 1 minuto, como podemos verificar por meio da Tabela 5.1. Nela temos todas as instâncias, exceto as densas, ordenadas por classe e tamanho, a quantidade (**Qtd**) de instâncias solucionadas (entre 0 e 10), e a média (**Avg**) do tempo de execução e o desvio padrão (**Std**), em segundos, das instâncias solucionadas de cada classe e tamanho. Em negrito temos os melhores resultados, em termos de mais instâncias solucionadas e menor tempo médio. Dessa forma, é possível pôr uma boa régua de comparação para os demais modelos. Entretanto, vamos averiguar mais adiante se os tempos de execução se mantêm aceitáveis quando comparados aos dos demais modelos.

É interessante notar que a variabilidade, medida pelo desvio padrão, foi baixa em ambos os modelos, indicando que qualquer instância dentro dos tamanhos e parâmetros

testados, com alta probabilidade, terá um tempo de execução bem próximo da média.

O modelo (DIB-F) obteve melhores tempos de execução para todas as instâncias testadas, indicando que a adição das restrições (4.8) ao modelo (DIB) o fortaleceu sem torná-lo pesado demais, e melhorou seu tempo de execução. Houve uma redução de aproximadamente 50% no tempo de execução para as instâncias da classe *Characteristics*, e de 15% a 30% para as do modelo *Neighborhood*.

Instância		DIB		DIB-F	
Classe	Tamanho (n)	Qtd	Avg $\pm$ Std	Qtd	Avg $\pm$ Std
<i>Characteristics</i>	100	10	1,76s $\pm$ 0,30s	10	0,70s $\pm$ 0,23s
<i>Characteristics</i>	150	10	2,23s $\pm$ 0,35s	10	0,74s $\pm$ 0,20s
<i>Characteristics</i>	300	10	2,39s $\pm$ 0,44s	10	1,33s $\pm$ 0,49s
<i>Characteristics</i>	500	10	7,17s $\pm$ 1,45s	10	3,80s $\pm$ 1,65s
<i>Characteristics</i>	1000	10	9,98s $\pm$ 0,90s	10	4,45s $\pm$ 0,75s
<i>Characteristics</i>	1500	10	20,45s $\pm$ 3,96s	10	9,06s $\pm$ 3,51s
<i>Characteristics</i>	2000	10	43,67s $\pm$ 8,20s	10	13,52s $\pm$ 3,40s
<i>Neighborhood</i>	100	10	1,54s $\pm$ 0,50s	10	0,98s $\pm$ 0,36s
<i>Neighborhood</i>	150	10	2,75s $\pm$ 1,07s	10	1,73s $\pm$ 0,49s
<i>Neighborhood</i>	300	10	6,64s $\pm$ 2,05s	10	4,82s $\pm$ 1,50s
<i>Neighborhood</i>	500	10	13,58s $\pm$ 3,08s	10	10,56s $\pm$ 2,03s
<i>Neighborhood</i>	1000	10	45,60s $\pm$ 13,52s	10	37,47s $\pm$ 14,83s
<i>Neighborhood</i>	1500	10	77,02s $\pm$ 8,62s	10	63,21s $\pm$ 8,35s
<i>Neighborhood</i>	2000	10	112,18s $\pm$ 19,82s	10	91,74s $\pm$ 17,97s
<i>Popularity</i>	100	10	9,03s $\pm$ 2,07s	10	4,11s $\pm$ 1,53s
<i>Popularity</i>	150	10	19,61s $\pm$ 3,79s	10	13,62s $\pm$ 2,72s
<i>Popularity</i>	300	10	139,19s $\pm$ 29,25s	10	103,39s $\pm$ 19,79s
<i>Popularity</i>	500	10	684,59s $\pm$ 163,32s	10	457,36s $\pm$ 104,79s
<i>Popularity</i>	1000	-	-	1	3.077,90s $\pm$ 0s
<i>Popularity</i>	1500	-	-	-	-
<i>Popularity</i>	2000	-	-	-	-

Tabela 5.1: Número de instâncias solucionadas e tempo de execução para os modelos (DIB) e (DIB-F).

Todas as instâncias solucionadas pelos modelos (DIB) e (DIB-F), exceto as instâncias da classe *Neighborhood* de tamanho 1000, tiveram suas soluções ótimas obtidas no nó raiz.

Seja z^* o valor da melhor solução obtida e $\bar{z}$ o valor da relaxação linear para uma instância qualquer. O Gap entre elas é definido por

$$\frac{\bar{z} - z^*}{z^*},$$

e na Tabela 5.2 temos todas as instâncias que foram solucionadas pelos modelos, ordenadas por classe e tamanho, e a média dos Gaps (**Gap (%)**) entre o valor da relaxação linear e o valor da solução ótima, com os menores Gaps médios em negrito, e o desvio padrão (**Std**). Embora o Gap médio seja pequeno mesmo para o modelo (DIB), podemos

constatar que para o modelo (DIB-F) ele é sempre menor. Isso pode ser apresentado como uma das causas do melhor desempenho do modelo fortalecido, que em várias instâncias, principalmente da classe *Characteristics*, obteve na relaxação o valor da solução ótima.

Portanto, o modelo (DIB-F) é realmente mais forte do que (DIB), embora seja mais pesado. A adição das restrições não atrapalhou o tempo de execução, que foi sempre melhor em (DIB-F) por conta da qualidade da sua relaxação.

Instância		DIB	DIB-F
Classe	Tamanho (n)	Gap (%) $\pm$ Std	Gap (%) $\pm$ Std
<i>Characteristic</i>	100	$1,442 \pm 0,628$	$0,048 \pm 0,062$
<i>Characteristic</i>	150	$0,868 \pm 0,490$	$0,005 \pm 0,015$
<i>Characteristic</i>	300	$0,138 \pm 0,108$	$0,007 \pm 0,015$
<i>Characteristic</i>	500	$0,177 \pm 0,126$	$0,015 \pm 0,035$
<i>Characteristic</i>	1000	$0,130 \pm 0,099$	$0,002 \pm 0,006$
<i>Characteristic</i>	1500	$0,333 \pm 0,112$	$0,004 \pm 0,005$
<i>Characteristic</i>	2000	$0,599 \pm 0,108$	$0,002 \pm 0,004$
<i>Neighborhood</i>	100	$1,549 \pm 0,955$	$0,660 \pm 0,453$
<i>Neighborhood</i>	150	$1,618 \pm 0,786$	$0,718 \pm 0,382$
<i>Neighborhood</i>	300	$1,434 \pm 0,441$	$0,733 \pm 0,377$
<i>Neighborhood</i>	500	$1,014 \pm 0,272$	$0,604 \pm 0,222$
<i>Neighborhood</i>	1000	$0,872 \pm 0,335$	$0,630 \pm 0,212$
<i>Neighborhood</i>	1500	$1,156 \pm 0,131$	$0,598 \pm 0,053$
<i>Neighborhood</i>	2000	$1,367 \pm 0,169$	$0,607 \pm 0,100$
<i>Popularity</i>	100	$4,699 \pm 1,810$	$0,168 \pm 0,189$
<i>Popularity</i>	150	$4,294 \pm 1,274$	$0,046 \pm 0,141$
<i>Popularity</i>	300	$4,484 \pm 1,049$	$0,023 \pm 0,039$
<i>Popularity</i>	500	$4,455 \pm 0,801$	$0,038 \pm 0,039$

Tabela 5.2: Gap médio entre valor da relaxação linear e valor da solução ótima para instâncias que foram solucionadas pelos modelos (DIB) e (DIB-F).

5.2.2 Modelos de Programação Linear Inteira Mista com Decisão Direta

Os modelos de PLIM, (DD) e (DD-F), obtiveram um resultado muito inferior aos modelos de PLI, conseguindo solucionar uma quantidade significativa apenas de instâncias menores, principalmente de tamanho 100 e 150, mas, ainda assim, com tempos de execução bem superiores, como podemos observar na Tabela 5.3.

O próprio fato do Gap médio ser maior, em comparação com os modelos de PLI, como podemos observar na Tabela 5.4, já seria uma justificativa para um maior tempo de execução dos modelos (DD) e (DD-F). Na Tabela 5.5, temos a média dos nós explorados (**Avg**) para que os modelos (DD) e (DD-F) conseguissem resolver as instâncias, além da mediana (**Med**), que nos fornece uma visão mais adequada devido à grande variabilidade dos dados, como podemos ver pelo desvio padrão (**Std**). Para essas poucas instâncias, os

Instância		DD		DD-F	
Classe	Tamanho (n)	Qtd	Avg $\pm$ Std	Qtd	Avg $\pm$ Std
<i>Characteristics</i>	100	10	985,32s $\pm$ 1.120,84s	10	710,82s $\pm$ 588,42s
<i>Characteristics</i>	150	4	1.596,67s $\pm$ 1.362,86s	4	1.623,13s $\pm$ 944,82s
<i>Characteristics</i>	300	-	-	-	-
<i>Characteristics</i>	500	-	-	-	-
<i>Characteristics</i>	1000	-	-	-	-
<i>Characteristics</i>	1500	-	-	-	-
<i>Characteristics</i>	2000	-	-	-	-
<i>Neighborhood</i>	100	10	12,82s $\pm$ 14,16s	10	18s $\pm$ 24,23s
<i>Neighborhood</i>	150	10	135,22s $\pm$ 183,24s	10	105,72s $\pm$ 119,79s
<i>Neighborhood</i>	300	8	784,16s $\pm$ 768,82s	8	774,16s $\pm$ 433,73s
<i>Neighborhood</i>	500	2	2.665,79s $\pm$ 412,92s	1	1.895,05s $\pm$ 0s
<i>Neighborhood</i>	1000	1	74,92s $\pm$ 0s	-	-
<i>Neighborhood</i>	1500	-	-	-	-
<i>Neighborhood</i>	2000	-	-	-	-
<i>Popularity</i>	100	9	1.581,63s $\pm$ 1.053,25s	9	1.512,96s $\pm$ 886,52s
<i>Popularity</i>	150	-	-	1	2.865,64s $\pm$ 0s
<i>Popularity</i>	300	-	-	-	-
<i>Popularity</i>	500	-	-	-	-
<i>Popularity</i>	1000	-	-	-	-
<i>Popularity</i>	1500	-	-	-	-
<i>Popularity</i>	2000	-	-	-	-

Tabela 5.3: Número de instâncias solucionadas e tempo de execução para os modelos (DD) e (DD-F).

modelos tiveram que explorar uma quantidade grande de nós. Nesses casos, as variáveis originalmente inteiras dificilmente acabam assumindo valores inteiros nas soluções das relaxações, levando à necessidade de se explorar mais nós na árvore de *branch-and-bound* para se obter uma solução ótima. Essa necessidade também pode ser vista como a causa de tão poucas instâncias terem sido resolvidas dentro do tempo limite.

Diante do exposto, as relaxações de (DD) e (DD-F) não são muito boas quando comparadas às dos modelos de PLI, e isso, unido a um *branch* ineficiente, fez com que os modelos obtivessem um desempenho ruim.

Quando comparamos os modelos (DD) e (DD-F) entre si, voltando novamente às Tabelas 5.3, 5.4, e 5.5, vemos que por conta das restrições adicionais o tempo médio de execução foi um pouco reduzido em alguns casos, principalmente se levarmos em conta o desvio padrão. A redução foi mais significativa para as instâncias da classe *Characteristics* de tamanho 100 e *Neighborhood* de tamanho 1000, mas para as últimas (DD-F) resolveu apenas uma instância e (DD) duas. Houve melhora na qualidade da relaxação para todas as instâncias, mas não em quantidade significativa, e quando olhamos para a média da quantidade de nós explorados, vemos que houve uma redução para a maior parte das instâncias. Mas isso pode ser contraposto à mediana, devido à variabilidade dos dados, indicando que não houve melhora. Portanto, podemos dizer que a adição das restrições (4.23) gerou um modelo mais forte que (DD), mas, na prática, não houve benefícios.

Instância		DD	DD-F
Classe	Tamanho (n)	Gap (%) $\pm$ Std	Gap (%) $\pm$ Std
<i>Characteristic</i>	100	$15,245 \pm 1,507$	$14,973 \pm 1,493$
<i>Characteristic</i>	150	$15,200 \pm 1,215$	$15,099 \pm 1,227$
<i>Neighborhood</i>	100	$13,868 \pm 3,111$	$13,724 \pm 2,966$
<i>Neighborhood</i>	150	$15,431 \pm 2,569$	$15,367 \pm 2,568$
<i>Neighborhood</i>	300	$16,679 \pm 1,425$	$16,679 \pm 1,425$
<i>Neighborhood</i>	500	$14,919 \pm 1,230$	$14,049 \pm 0$
<i>Popularity</i>	100	$18,250 \pm 2,120$	$18,701 \pm 3,269$

Tabela 5.4: Gap médio entre valor da relaxação linear e valor da solução ótima para instâncias que foram solucionadas pelos modelos (DD) e (DD-F).

Instância		DD		DD-F	
Classe	Tamanho (n)	Avg $\pm$ Std	Med	Avg $\pm$ Std	Med
<i>Characteristic</i>	100	10.709 ± 9.817	8.118	9.049 ± 6.881	9.208
<i>Characteristic</i>	150	9.014 ± 7.068	6.313	9.015 ± 3.273	8.998
<i>Neighborhood</i>	100	921 ± 1.200	633	1.208 ± 1.884	426
<i>Neighborhood</i>	150	4.266 ± 5.053	1.916	3.692 ± 4.546	1.888
<i>Neighborhood</i>	300	24.603 ± 27.275	12.551	17.995 ± 10.711	21.672
<i>Neighborhood</i>	500	45.911 ± 6.740	45.911	28.929 ± 0	28.929
<i>Popularity</i>	100	21.921 ± 15.947	20.931	15.433 ± 8.236	20.530

Tabela 5.5: Média e mediana de nós explorados para instâncias que foram solucionadas pelos modelos (DD) e (DD-F).

5.2.3 Modelo de Programação Linear Inteira Mista com Decisão Indireta

Os modelos (DIV) e (DIV-F) tiveram como ponto de partida o modelo (DD), mas com a remoção das variáveis x_{ib} e $\hat{p}_{ib}$ e a adição das variáveis $\hat{p}_b$, e também com mudanças nas restrições. Com um novo conjunto de variáveis, temos a possibilidade de uma melhor relaxação e um *branch* mais efetivo.

Se compararmos inicialmente (DIV) com (DIV-F), por meio da Tabela 5.6, antes de os compararmos com (DD) e (DD-F), iremos perceber que a adição das restrições (4.23) foi benéfica em relação ao tempo de execução para todas as instâncias solucionadas pelos modelos, mas não necessariamente significativa em todos os casos, com exceção de uma única instância da classe *Neighborhood* de tamanho 1000 em que (DIV) obteve melhor resultado. Além disso, uma quantidade maior de instâncias foi solucionada por (DIV-F), como no caso de instâncias de tamanho 500 para as classes *Characteristics* e *Neighborhood*. Olhando para o Gap médio, na Tabela 5.7, percebemos que o modelo (DIV-F) teve um Gap médio menor para todas as instâncias, indicando uma melhor relaxação. Portanto, isso nos leva a concluir que o (DIV-F) é realmente mais forte que (DIV), e isso trouxe uma

melhora nos resultados de tempo de execução e quantidade de instâncias solucionadas.

Instância		DIV		DIV-F	
Classe	Tamanho (n)	Qtd	Avg $\pm$ Std	Qtd	Avg $\pm$ Std
<i>Characteristics</i>	100	10	23,99s $\pm$ 22,96s	10	13,41s $\pm$ 12,89s
<i>Characteristics</i>	150	10	46,50s $\pm$ 40,83s	10	34,16s $\pm$ 33,18s
<i>Characteristics</i>	300	10	175,55s $\pm$ 98,43s	10	161,89s $\pm$ 88,11s
<i>Characteristics</i>	500	8	1.170,42s $\pm$ 737,31s	9	936,26s $\pm$ 499,94s
<i>Characteristics</i>	1000	10	1.527,91s $\pm$ 270,92s	10	1.258,56s $\pm$ 273,08s
<i>Characteristics</i>	1500	3	3.190,09s $\pm$ 197,81s	6	2.906,38s $\pm$ 522,46s
<i>Characteristics</i>	2000	-	-	-	-
<i>Neighborhood</i>	100	10	5,72s $\pm$ 6,31s	10	4,10s $\pm$ 4,42s
<i>Neighborhood</i>	150	10	22,08s $\pm$ 26,31s	10	13,73s $\pm$ 12,01s
<i>Neighborhood</i>	300	10	418,19s $\pm$ 845,95s	10	307,32s $\pm$ 543,96s
<i>Neighborhood</i>	500	8	957,19s $\pm$ 986,59s	10	947,58s $\pm$ 885,69s
<i>Neighborhood</i>	1000	1	1.895,14s $\pm$ 0s	1	2.897,94s $\pm$ 0s
<i>Neighborhood</i>	1500	-	-	-	-
<i>Neighborhood</i>	2000	-	-	-	-
<i>Popularity</i>	100	10	43,03s $\pm$ 30,98s	10	34,02s $\pm$ 26,61s
<i>Popularity</i>	150	10	392,42s $\pm$ 519,84s	10	348,19s $\pm$ 563,87s
<i>Popularity</i>	300	3	1.749,99s $\pm$ 941,36s	4	923,34s $\pm$ 709,07s
<i>Popularity</i>	500	-	-	-	-
<i>Popularity</i>	1000	-	-	-	-
<i>Popularity</i>	1500	-	-	-	-
<i>Popularity</i>	2000	-	-	-	-

Tabela 5.6: Número de instâncias solucionadas e tempo de execução para os modelos (DIV) e (DIV-F).

Instância		DIV	DIV-F
Classe	Tamanho (n)	Gap (%) $\pm$ Std	Gap (%) $\pm$ Std
<i>Characteristic</i>	100	10,968 $\pm$ 1,511	8,594 $\pm$ 1,270
<i>Characteristic</i>	150	10,625 $\pm$ 1,562	8,130 $\pm$ 1,181
<i>Characteristic</i>	300	10,356 $\pm$ 0,920	7,752 $\pm$ 0,918
<i>Characteristic</i>	500	11,182 $\pm$ 0,694	8,350 $\pm$ 0,579
<i>Characteristic</i>	1000	10,433 $\pm$ 0,494	7,587 $\pm$ 0,384
<i>Characteristic</i>	1500	10,306 $\pm$ 0,352	7,720 $\pm$ 0,310
<i>Neighborhood</i>	100	9,333 $\pm$ 2,321	7,042 $\pm$ 2,367
<i>Neighborhood</i>	150	9,169 $\pm$ 2,224	6,689 $\pm$ 1,963
<i>Neighborhood</i>	300	10,984 $\pm$ 1,629	7,950 $\pm$ 1,671
<i>Neighborhood</i>	500	9,595 $\pm$ 1,882	7,074 $\pm$ 1,490
<i>Neighborhood</i>	1000	9,087 $\pm$ 0	6,243 $\pm$ 0
<i>Popularity</i>	100	15,359 $\pm$ 2,783	14,167 $\pm$ 2,430
<i>Popularity</i>	150	16,290 $\pm$ 2,177	15,105 $\pm$ 2,199
<i>Popularity</i>	300	16,845 $\pm$ 0,694	15,066 $\pm$ 0,946

Tabela 5.7: Gap médio entre valor da relaxação linear e valor da solução ótima para instâncias que foram solucionadas pelos modelos (DIV) e (DIV-F).

Agora, iniciando a comparação entre os modelos de PLIM, se analisarmos a quantidade de instâncias solucionadas pelo modelo (DIV-F) em relação ao modelo (DD-F) (já que (DD) e (DD-F) obtiveram resultados equivalentes), por meio de um paralelo entre as Tabelas 5.3 e 5.6, veremos que o (DIV-F) conseguiu solucionar uma quantidade muito maior de instâncias, principalmente da classe *Characteristics*, e os tempos de execução médios sempre foram melhores.

Comparando, na Tabela 5.8, o Gap médio para os modelos (DD-F) e (DIV-F), para instâncias pequenas que foram as que (DD-F) conseguiu resolver, vemos que o Gap médio de (DD-F) foi de 74% a 130% maior que o de (DIV-F) para instâncias das classes *Characteristics* e *Neighborhood*, e essa diferença foi um pouco menos evidente para a classe *Popularity*, sendo de 17% a 32% maior. Então (DIV-F) realmente tem uma relaxação melhor, e isso contribuiu para o seu melhor desempenho.

Instância		DD-F	DIV-F
Classe	Tamanho (n)	Gap (%) $\pm$ Std	Gap (%) $\pm$ Std
<i>Characteristic</i>	100	14,973 $\pm$ 1,493	8,594 $\pm$ 1,270
<i>Characteristic</i>	150	15,099 $\pm$ 1,227	8,130 $\pm$ 1,181
<i>Neighborhood</i>	100	13,724 $\pm$ 2,966	7,042 $\pm$ 2,367
<i>Neighborhood</i>	150	15,367 $\pm$ 2,568	6,689 $\pm$ 1,963
<i>Neighborhood</i>	300	16,680 $\pm$ 1,425	7,950 $\pm$ 1,671
<i>Neighborhood</i>	500	14,049 $\pm$ 0	7,074 $\pm$ 1,490
<i>Popularity</i>	100	18,701 $\pm$ 3,269	14,167 $\pm$ 2,430
<i>Popularity</i>	150	17,711 $\pm$ 0	15,105 $\pm$ 2,199

Tabela 5.8: Gap médio entre valor da relaxação linear e valor da solução ótima para instâncias que foram solucionadas pelos modelos (DD-F) e (DIV-F).

Buscando entender ainda outros motivos que levaram a uma diferença de performance tão grande entre os modelos, podemos tomar como ponto de partida um exemplo do progresso do valor da função objetivo para uma instância da classe *Characteristics* de tamanho 100, como apresentado nas Figuras 5.1 e 5.2, que teve um tempo de execução próximo das respectivas médias para os modelos (DD-F) e (DIV-F), que foram de 710,82s e 13,41s. Observamos uma rápida aproximação entre o valor da melhor solução inteira e do melhor limitante superior (*bound*) para (DD-F), mas, após isso, uma lenta convergência, enquanto que para (DIV-F) essa convergência ocorre em um intervalo de poucos segundos.

Isso é um forte indício, em conjunto com a quantidade de nós explorados, de que (DD-F) tem dificuldade em definir valores inteiros para as variáveis em suas soluções. Quanto mais fracionárias forem as soluções, possivelmente, maior terá que ser a árvore de *branch-and-bound* se o *branch* também for ruim. Isso significa que um longo caminho terá que ser percorrido para converter as soluções fracionárias em soluções inteiras, tomando um tempo maior para a convergência entre soluções viáveis e limitantes superiores.

Analisando a quantidade de nós explorados para as instâncias que ambos os modelos conseguiram solucionar, na Tabela 5.9, notamos que (DD-F) realmente explorou uma

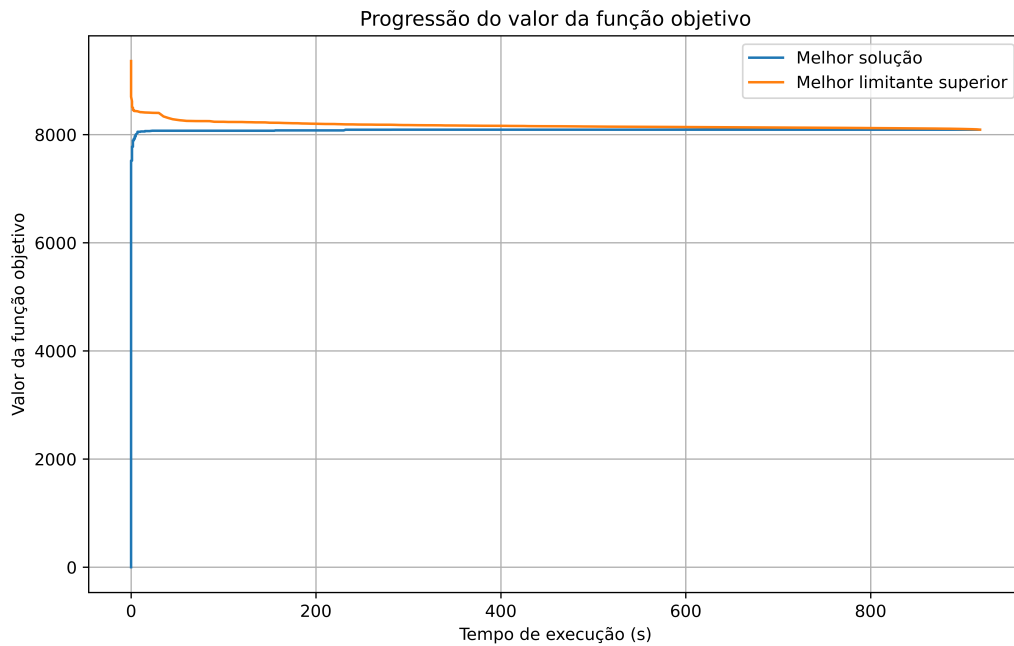


Figura 5.1: Progresso dos valores de melhor solução e melhor *bound* para instância da classe *Characteristics* de tamanho 100, para o modelo (DD-F).

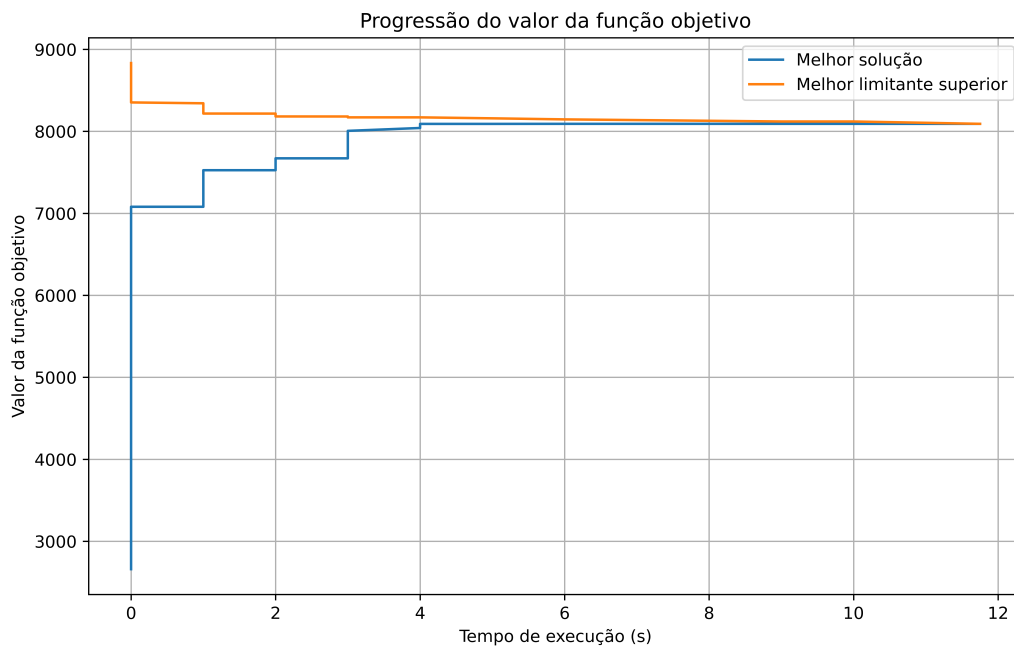


Figura 5.2: Progresso dos valores de melhor solução e melhor *bound* para instância da classe *Characteristics* de tamanho 100, para o modelo (DIV-F).

quantidade muito grande de nós em comparação com (DIV-F), principalmente se analisarmos a mediana, indicando um *branch* bem menos eficaz, que gera subproblemas que não reduzem de forma significativa o espaço de busca.

Instância		DD-F		DIV-F	
Classe	Tamanho (n)	Avg $\pm$ Std	Med	Avg $\pm$ Std	Med
<i>Characteristic</i>	100	9.049 ± 6.881	9.208	362 ± 399	252
<i>Characteristic</i>	150	9.014 ± 3.273	8.998	654 ± 574	487
<i>Neighborhood</i>	100	1.208 ± 1.884	426	620 ± 827	102
<i>Neighborhood</i>	150	3.692 ± 4.546	1.888	1.083 ± 892	729
<i>Neighborhood</i>	300	17.995 ± 10.711	21.672	8.229 ± 9.706	5.065
<i>Neighborhood</i>	500	28.929 ± 0	28.929	16.724 ± 11.070	15.161
<i>Popularity</i>	100	15.433 ± 8.236	20.530	1.168 ± 655	1.412
<i>Popularity</i>	150	13.963 ± 0	13.963	4.611 ± 8.177	1345

Tabela 5.9: Média e mediana de nós explorados para instâncias que foram solucionadas pelos modelos (DD-F) e (DIV-F).

A quantidade de variáveis pode ter desempenhado um impacto significativo na qualidade do *branch*, já que quanto mais variáveis, maior o espaço de busca, e isso aumenta a dificuldade de se selecionar uma boa variável para dividir esse espaço. Comparando o tamanho dos modelos (DIV-F) e (DD-F), tanto em um quanto em outro temos assintoticamente $\Theta(\|v\|_0 + |I|)$ variáveis, mas, na prática, o número exato de variáveis é um pouco discrepante. Com base na Tabela 5.10, onde encontramos o número médio de variáveis inteiras (**Vars**) após o *presolve* do *Gurobi*, que consiste em uma etapa de pré-processamento que visa simplificar o modelo e torná-lo mais fácil de resolver, com as menores quantidades de variáveis em negrito, vemos que (DD-F) sempre tem, na média, mais do que o dobro da quantidade de variáveis inteiras de (DIV-F).

Instância		DD-F	DIV-F
Classe	Tamanho (n)	Vars $\pm$ Std	Vars $\pm$ Std
<i>Characteristic</i>	100	$1.606 \pm 33,32$	800,10 $\pm$ 18,33
<i>Characteristic</i>	150	$2.362 \pm 52,40$	1.173,10 $\pm$ 28,55
<i>Neighborhood</i>	100	$1.422,70 \pm 41,06$	706,10 $\pm$ 20,56
<i>Neighborhood</i>	150	$2.117,80 \pm 127$	1.053 $\pm$ 63,84
<i>Neighborhood</i>	300	$4.398,25 \pm 77,54$	2.189 $\pm$ 37,71
<i>Neighborhood</i>	500	7.628 ± 0	3.779 $\pm$ 94,62
<i>Popularity</i>	100	1.600 ± 0	774,50 $\pm$ 7,62
<i>Popularity</i>	150	2.400 ± 0	1.169,10 $\pm$ 6,54

Tabela 5.10: Número médio de variáveis inteiras após o *presolve* para instâncias que foram solucionadas pelos modelos (DD-F) e (DIV-F).

Dado o que foi exposto, (DIV-F) tem uma melhor relaxação que (DD-F) e menos variáveis, o que colaborou para um *branch* mais eficaz e uma menor enumeração de nós, levando (DIV-F) a obter resultados bem mais interessantes.

Sendo (DIV-F) o melhor modelo de PLIM até então, podemos realizar algumas análises

comparativas entre ele e (DIB-F), que obteve a melhor performance para PLI. O modelo de PLI conseguiu solucionar um número maior de instâncias, como podemos comparar na Tabela 5.11. Foram solucionadas todas as instâncias das classes *Characteristics* e *Neighborhood*, enquanto o modelo (DIV-F) solucionou menos instâncias para tamanhos um pouco maiores. O crescimento do tempo de execução conforme o tamanho das instâncias aumenta foi mais acentuado para (DIV-F); isto é, o aumento do número de compradores ou itens gera um impacto maior em (DIV-F) do que em (DIB-F).

Instância		DIV-F		DIB-F	
Classe	Tamanho (n)	Qtd	Avg $\pm$ Std	Qtd	Avg $\pm$ Std
<i>Characteristics</i>	100	10	13,41s $\pm$ 12,89s	10	0,70s $\pm$ 0,23s
<i>Characteristics</i>	150	10	34,16s $\pm$ 33,18s	10	0,74s $\pm$ 0,20s
<i>Characteristics</i>	300	10	161,89s $\pm$ 88,11s	10	1,33s $\pm$ 0,49s
<i>Characteristics</i>	500	9	936,26s $\pm$ 499,94s	10	3,80s $\pm$ 1,65s
<i>Characteristics</i>	1000	10	1.258,56s $\pm$ 273,08s	10	4,45s $\pm$ 0,75s
<i>Characteristics</i>	1500	6	2.906,38s $\pm$ 522,46s	10	9,06s $\pm$ 3,51s
<i>Neighborhood</i>	100	10	4,10s $\pm$ 4,42s	10	0,98s $\pm$ 0,36s
<i>Neighborhood</i>	150	10	13,73s $\pm$ 12,01s	10	1,73s $\pm$ 0,49s
<i>Neighborhood</i>	300	10	307,32s $\pm$ 543,96s	10	4,82s $\pm$ 1,50s
<i>Neighborhood</i>	500	10	947,58s $\pm$ 885,69s	10	10,56s $\pm$ 2,03s
<i>Neighborhood</i>	1000	1	2.897,94s $\pm$ 0s	10	37,47s $\pm$ 14,83s
<i>Popularity</i>	100	10	34,02s $\pm$ 26,61s	10	4,11s $\pm$ 1,53s
<i>Popularity</i>	150	10	348,19s $\pm$ 563,87s	10	13,62s $\pm$ 2,72s
<i>Popularity</i>	300	4	923,34s $\pm$ 709,07s	10	103,39s $\pm$ 19,79s

Tabela 5.11: Número de instâncias solucionadas e tempo de execução para os modelos (DIV-F) e (DIB-F).

Se lembrarmos que (DIB-F) tem uma relaxação linear com valor muito próximo do valor da solução ótima, e que para praticamente todas as instâncias conseguiu obter a solução ótima no nó raiz, a diferença de performance se torna compreensível, já que (DIV-F) tem uma relaxação com valor bem mais distante do valor da solução ótima, e por isso precisa explorar mais nós para solucionar as instâncias. Quando a relaxação linear já está próxima da solução ótima, o esforço necessário para encontrá-la é consideravelmente reduzido. Um Gap pequeno sugere que menos nós serão explorados até se encontrar uma solução inteira, acelerando o processo de exploração da árvore de *branch-and-bound*.

5.2.4 Modelo de Programação Linear Inteira Mista com Decisão Direta e Por Inveja

O modelo (DDI) é o modelo com o menor número de variáveis e restrições para instâncias esparsas, na prática, dentre os modelos de PLIM, como podemos observar na Tabela 5.12, onde temos o número médio de variáveis (**Vars**) e restrições (**Constrs**) para os modelos

(DIV-F) e (DDI) após o *presolve*, com as menores quantidades de variáveis e restrições em negrito. Isso levanta a possibilidade de um melhor desempenho por conta de (DDI) ser mais leve, ou talvez pior, se for mais fraco.

Instância		DIV-F		DDI	
Classe	Tamanho (n)	Vars $\pm$ Std	Constrs $\pm$ Std	Vars $\pm$ Std	Constrs $\pm$ Std
<i>Characteristic</i>	100	800,10 $\pm$ 18,33	3.192,80 $\pm$ 74,22	409,10 $\pm$ 21,46	1.656,60 $\pm$ 102,04
<i>Characteristic</i>	150	1.173,10 $\pm$ 28,55	4.690,70 $\pm$ 112,75	609,30 $\pm$ 32,31	2.526,60 $\pm$ 181,81
<i>Characteristic</i>	300	2.115,90 $\pm$ 31,42	8.427,60 $\pm$ 128,74	1.085,40 $\pm$ 42,99	4.417,60 $\pm$ 206,96
<i>Characteristic</i>	500	3.988 $\pm$ 58,31	15.777,22 $\pm$ 363,67	2.054,70 $\pm$ 46,19	9.123,90 $\pm$ 337,52
<i>Characteristic</i>	1000	7.149,80 $\pm$ 62,66	27.816,80 $\pm$ 384,27	3.682,70 $\pm$ 50,79	15.333,40 $\pm$ 258,26
<i>Characteristic</i>	1500	10.740,80 $\pm$ 127,30	41.680,50 $\pm$ 511,73	5.547,30 $\pm$ 103	23.277 $\pm$ 607,34
<i>Neighborhood</i>	100	706,10 $\pm$ 20,56	2.821,30 $\pm$ 81,56	221,50 $\pm$ 19,28	727,10 $\pm$ 93,98
<i>Neighborhood</i>	150	1.053 $\pm$ 63,84	4.205,90 $\pm$ 255,55	332,40 $\pm$ 30,93	1.091,10 $\pm$ 101,60
<i>Neighborhood</i>	300	2.189 $\pm$ 37,71	8.723,30 $\pm$ 148,96	733,60 $\pm$ 38,93	2.448,30 $\pm$ 82,69
<i>Neighborhood</i>	500	3.779 $\pm$ 94,62	14.989,70 $\pm$ 422,19	1.239,30 $\pm$ 54,99	4.150,90 $\pm$ 156,93
<i>Neighborhood</i>	1000	7.592 $\pm$ 0	29.904 $\pm$ 0	2.608,20 $\pm$ 59,10	8.954,40 $\pm$ 193,35
<i>Popularity</i>	100	774,50 $\pm$ 7,62	3.089 $\pm$ 25,66	381,20 $\pm$ 16,98	2.510,50 $\pm$ 303,48
<i>Popularity</i>	150	1.169,10 $\pm$ 6,54	4.656,90 $\pm$ 28,61	569,40 $\pm$ 36,87	4.745 $\pm$ 553,18
<i>Popularity</i>	300	2.349,50 $\pm$ 12,87	9.362,25 $\pm$ 51,11	1.176,40 $\pm$ 53,24	15.579,89 $\pm$ 1.567,44

Tabela 5.12: Número médio de variáveis inteiras e restrições após o *presolve* para instâncias que foram solucionadas pelos modelos (DIV-F) e (DDI).

Na Tabela 5.13 podemos ver que o desempenho de (DDI) foi sim melhor, pois conseguiu solucionar um número maior de instâncias que (DIV-F) – praticamente a mesma quantidade que (DIB-F) –, com um tempo de execução médio menor. Observando a Tabela 5.14, verificamos que o Gap médio para (DDI) foi consideravelmente menor que para (DIV-F), pois o Gap médio de (DIV-F) foi pelo menos o dobro do de (DDI), para a maior parte dos casos. É interessante o fato de que um modelo mais leve tenha gerado uma relaxação melhor, e isso indica (DDI) é um modelo mais forte que (DIV-F). Portanto, sendo mais leve e mais forte, (DDI) obteve o melhor resultado para os modelos de PLIM.

Também é interessante notarmos a relação entre a classe da instância e o valor do Gap médio da relaxação. Instâncias da classe *Characteristics* tiveram um Gap médio próximo de 4%, da classe *Neighborhood* próximo de 1%, e da classe *Popularity* próximo de 5% ou 6%, independentemente do tamanho da instância. Portanto, a relaxação de (DDI) é sensível à classe da instância, diferente do que ocorreu com os outros modelos de PLIM.

Instância		DIV-F		DDI	
Classe	Tamanho (<i>n</i>)	Qtd	Avg $\pm$ Std	Qtd	Avg $\pm$ Std
<i>Characteristics</i>	100	10	13,41s $\pm$ 12,89s	10	0,95s $\pm$ 0,38s
<i>Characteristics</i>	150	10	34,16s $\pm$ 33,18s	10	4,77s $\pm$ 8,40s
<i>Characteristics</i>	300	10	161,89s $\pm$ 88,11s	10	26,31s $\pm$ 29,36s
<i>Characteristics</i>	500	9	936,26s $\pm$ 499,94s	10	273,90s $\pm$ 297,54s
<i>Characteristics</i>	1000	10	1.258,56s $\pm$ 273,08s	10	220,98s $\pm$ 108,14s
<i>Characteristics</i>	1500	6	2.906,38s $\pm$ 522,46s	9	890,57s $\pm$ 567,76s
<i>Characteristics</i>	2000	-	-	6	1.498,30s $\pm$ 903,68s
<i>Neighborhood</i>	100	10	4,10s $\pm$ 4,42s	10	0,11s $\pm$ 0,03s
<i>Neighborhood</i>	150	10	13,73s $\pm$ 12,01s	10	0,21s $\pm$ 0,05s
<i>Neighborhood</i>	300	10	307,32s $\pm$ 543,96s	10	0,55s $\pm$ 0,21s
<i>Neighborhood</i>	500	10	947,58s $\pm$ 885,69s	10	1,08s $\pm$ 0,41s
<i>Neighborhood</i>	1000	1	2.897,94s $\pm$ 0s	10	3,98s $\pm$ 1,38s
<i>Neighborhood</i>	1500	-	-	10	5,64s $\pm$ 1,11s
<i>Neighborhood</i>	2000	-	-	10	8,50s $\pm$ 1,99s
<i>Popularity</i>	100	10	34,02s $\pm$ 26,61s	10	9,54s $\pm$ 21,01s
<i>Popularity</i>	150	10	348,19s $\pm$ 563,87s	10	45,49s $\pm$ 83,51s
<i>Popularity</i>	300	4	923,34s $\pm$ 709,07s	9	885,87s $\pm$ 983,27s
<i>Popularity</i>	500	-	-	1	361,83s $\pm$ 0s
<i>Popularity</i>	1000	-	-	-	-
<i>Popularity</i>	1500	-	-	-	-
<i>Popularity</i>	2000	-	-	-	-

Tabela 5.13: Número de instâncias solucionadas e tempo de execução para os modelos (DIV-F) e (DDI).

Instância		DIV-F	DDI
Classe	Tamanho (<i>n</i>)	Gap (%) $\pm$ Std	Gap (%) $\pm$ Std
<i>Characteristic</i>	100	8,594 $\pm$ 1,270	3,890 $\pm$ 0,825
<i>Characteristic</i>	150	8,130 $\pm$ 1,181	3,938 $\pm$ 0,843
<i>Characteristic</i>	300	7,752 $\pm$ 0,918	3,919 $\pm$ 0,678
<i>Characteristic</i>	500	8,350 $\pm$ 0,579	4,383 $\pm$ 0,563
<i>Characteristic</i>	1000	7,587 $\pm$ 0,384	3,824 $\pm$ 0,326
<i>Characteristic</i>	1500	7,720 $\pm$ 0,310	3,962 $\pm$ 0,252
<i>Neighborhood</i>	100	7,042 $\pm$ 2,367	0,978 $\pm$ 0,545
<i>Neighborhood</i>	150	6,689 $\pm$ 1,963	1,258 $\pm$ 0,569
<i>Neighborhood</i>	300	7,950 $\pm$ 1,671	1,213 $\pm$ 0,370
<i>Neighborhood</i>	500	7,074 $\pm$ 1,490	1,070 $\pm$ 0,309
<i>Neighborhood</i>	1000	6,243 $\pm$ 0	1,190 $\pm$ 0,209
<i>Popularity</i>	100	14,167 $\pm$ 2,430	4,661 $\pm$ 1,433
<i>Popularity</i>	150	15,105 $\pm$ 2,199	5,295 $\pm$ 1,291
<i>Popularity</i>	300	15,066 $\pm$ 0,946	6,075 $\pm$ 0,933
<i>Popularity</i>	500	14,919 $\pm$ 0	5,224 $\pm$ 0

Tabela 5.14: Gap médio entre valor da relaxação linear e valor da solução ótima para instâncias solucionadas pelos modelos (DIV-F) e (DDI).

Ainda sobre o melhor desempenho de (DDI), olhando para a Tabela 5.15 vemos que a quantidade de nós explorados por (DDI) foi bem menor que a de (DIV-F) para instâncias das classes *Characteristics* e *Popularity* de tamanho 100 e 150, e da classe *Neighborhood*. E isso faz sentido, pois o tempo de execução de (DDI) para esses casos também foi bem menor. Para os demais casos, a quantidade de nós explorados foi equivalente, mas o tempo de execução foi menor. Lembrando que (DDI) é um modelo bem mais leve que (DIV-F), então esses casos também fazem sentido.

Instância		DIV-F		DDI	
Classe	Tamanho (n)	Avg $\pm$ Std	Med	Avg $\pm$ Std	Med
<i>Characteristic</i>	100	362,30 $\pm$ 399,39	251,50	68 $\pm$ 84	35
<i>Characteristic</i>	150	654,40 $\pm$ 573,70	487	214 $\pm$ 472	2
<i>Characteristic</i>	300	962,70 $\pm$ 892,92	722	929 $\pm$ 1.040	531
<i>Characteristic</i>	500	1.312,33 $\pm$ 844,04	690	1.401 $\pm$ 1.096	1.160
<i>Characteristic</i>	1000	626,70 $\pm$ 69,19	610,50	720 $\pm$ 264	625
<i>Characteristic</i>	1500	708,67 $\pm$ 203,83	640	1.342 $\pm$ 810	1.021
<i>Characteristic</i>	2000	—	—	1.121 $\pm$ 762	777
<i>Neighborhood</i>	100	620 $\pm$ 827,09	101,50	1 $\pm$ 0	1
<i>Neighborhood</i>	150	1.083,40 $\pm$ 891,95	729	1 $\pm$ 0	1
<i>Neighborhood</i>	300	8.229,20 $\pm$ 9.706,35	5.065	1,20 $\pm$ 0,63	1
<i>Neighborhood</i>	500	16.723,90 $\pm$ 11.070,48	15.160,50	1 $\pm$ 0	1
<i>Neighborhood</i>	1000	40.401 $\pm$ 0	40.401	1 $\pm$ 0	1
<i>Neighborhood</i>	1500	—	—	1 $\pm$ 0	1
<i>Neighborhood</i>	2000	—	—	1 $\pm$ 0	1
<i>Popularity</i>	100	1.167,60 $\pm$ 655,36	1.411,50	438 $\pm$ 844	86
<i>Popularity</i>	150	4.610,60 $\pm$ 8.177	1.345	683 $\pm$ 895	201
<i>Popularity</i>	300	2.424,75 $\pm$ 2.122,08	1.630,50	2.099 $\pm$ 1.568	1.225
<i>Popularity</i>	500	—	—	854 $\pm$ 0	854

Tabela 5.15: Média e mediana de nós explorados para instâncias que foram solucionadas pelos modelo (DIV-F) e (DDI).

É interessante o fato de que para a maioria das instâncias da classe *Neighborhood* a solução foi encontrada somente por meio da exploração do nó raiz. E esses foram justamente os casos em que (DDI) obteve as melhores relaxações. Isso indica que o modelo tem mais dificuldade em encontrar boas soluções viáveis do que bons limitantes.

Ao compararmos os resultados de (DDI) com (DIB-F), na Tabela 5.16, que tinha sido o melhor modelo até então, percebemos uma clara segmentação por classe de instância: (DIB-F) foi melhor para as instâncias da classe *Characteristics* e *Popularity*, e (DDI) para as da classe *Neighborhood*. Aqui, podemos fazer um paralelo entre as relaxações. Como a relaxação de (DIB-F) é muito boa, como podemos ver novamente na Tabela 5.17, ele conseguiu solucionar todas as instâncias, exceto as instâncias da classe *Neighborhood* de tamanho 1000, explorando somente o nó raiz. Para as instâncias da classe *Neighborhood*, onde a relaxação de (DDI) também foi muito boa, ele obteve melhores resultados que (DIB-F). Ou seja, quando ambas as relaxações são muito boas, sendo possível solucionar

as instâncias somente com o nó raiz, podemos entender que (DDI) foi melhor pelo fato de ser mais leve.

Instância		DIB-F		DDI	
Classe	Tamanho (n)	Qtd	Avg $\pm$ Std	Qtd	Avg $\pm$ Std
<i>Characteristics</i>	100	10	0,70s $\pm$ 0,23s	10	0,95s $\pm$ 0,38s
<i>Characteristics</i>	150	10	0,74s $\pm$ 0,20s	10	4,77s $\pm$ 8,40s
<i>Characteristics</i>	300	10	1,33s $\pm$ 0,49s	10	26,31s $\pm$ 29,36s
<i>Characteristics</i>	500	10	3,80s $\pm$ 1,65s	10	273,90s $\pm$ 297,54s
<i>Characteristics</i>	1000	10	4,45s $\pm$ 0,75s	10	220,98s $\pm$ 108,14s
<i>Characteristics</i>	1500	10	9,06s $\pm$ 3,51s	9	890,57s $\pm$ 567,76s
<i>Characteristics</i>	2000	10	13,52s $\pm$ 3,40s	6	1.498,30s $\pm$ 903,68s
<i>Neighborhood</i>	100	10	0,98s $\pm$ 0,36s	10	0,11s $\pm$ 0,03s
<i>Neighborhood</i>	150	10	1,73s $\pm$ 0,49s	10	0,21s $\pm$ 0,05s
<i>Neighborhood</i>	300	10	4,82s $\pm$ 1,50s	10	0,55s $\pm$ 0,21s
<i>Neighborhood</i>	500	10	10,56s $\pm$ 2,03s	10	1,08s $\pm$ 0,41s
<i>Neighborhood</i>	1000	10	37,47s $\pm$ 14,83s	10	3,98s $\pm$ 1,38s
<i>Neighborhood</i>	1500	10	63,21s $\pm$ 8,35s	10	5,64s $\pm$ 1,11s
<i>Neighborhood</i>	2000	10	91,74s $\pm$ 17,97s	10	8,50s $\pm$ 1,99s
<i>Popularity</i>	100	10	4,11s $\pm$ 1,53s	10	9,54s $\pm$ 21,01s
<i>Popularity</i>	150	10	13,62s $\pm$ 2,72s	10	45,49s $\pm$ 83,51s
<i>Popularity</i>	300	10	103,39s $\pm$ 19,79s	9	885,87s $\pm$ 983,27s
<i>Popularity</i>	500	10	457,36s $\pm$ 104,79s	1	361,83s $\pm$ 0s

Tabela 5.16: Número de instâncias solucionadas e tempo de execução para os modelos (DIB-F) e (DDI).

5.3 Instâncias Densas

Quando analisamos os resultados para as instâncias densas, em que todo comprador oferece uma valoração para todo item, percebemos que o desempenho de alguns modelos se altera. Analisando a Tabela 5.18 e iniciando pelos modelos de PLI, que obtiveram o melhor desempenho para as instâncias esparsas, notamos que eles conseguiram solucionar apenas as instâncias densas de tamanho 100, e ainda com um tempo elevado.

O modelo (DIB) teve um alto tempo de pré-processamento e também gastou um tempo considerável para obter o valor da relaxação, e esses problemas apenas se agravaram para o modelo (DIB-F), que possui mais restrições. Para as instâncias maiores, não foi possível carregar os modelos em memória por conta de o modelo ser muito grande.

Os modelos de PLIM (DIV), (DIV-F) e (DDI) obtiveram bons resultados, conseguindo solucionar todas as instâncias. É interessante notar que a adição de restrições ao modelo (DIV), que gerou (DIV-F), não foi benéfica nesse caso, elevando o tempo de execução. Isso demonstra que essas instâncias são relativamente fáceis de se resolver, e a adição de restrições tornou o modelo apenas mais pesado e lento, pois a adição das restrições não trouxe melhora na relaxação para o caso das instâncias densas.

Quando analisamos os resultados do modelo (DDI), percebemos um aspecto positivo e outro negativo. O positivo é que o modelo também conseguiu solucionar todas as instâncias, e o negativo é que seu desempenho ficou abaixo do desempenho de (DIV),

Instância		DIB-F	DDI
Classe	Tamanho (n)	Gap (%) $\pm$ Std	Gap (%) $\pm$ Std
<i>Characteristic</i>	100	0,048 $\pm$ 0,062	3,890 $\pm$ 0,825
<i>Characteristic</i>	150	0,005 $\pm$ 0,015	3,938 $\pm$ 0,843
<i>Characteristic</i>	300	0,007 $\pm$ 0,015	3,919 $\pm$ 0,678
<i>Characteristic</i>	500	0,015 $\pm$ 0,035	4,383 $\pm$ 0,563
<i>Characteristic</i>	1000	0,002 $\pm$ 0,006	3,824 $\pm$ 0,326
<i>Characteristic</i>	1500	0,004 $\pm$ 0,005	3,962 $\pm$ 0,252
<i>Characteristic</i>	2000	0,002 $\pm$ 0,004	3,919 $\pm$ 0,212
<i>Neighborhood</i>	100	0,660 $\pm$ 0,453	0,978 $\pm$ 0,545
<i>Neighborhood</i>	150	0,718 $\pm$ 0,382	1,258 $\pm$ 0,569
<i>Neighborhood</i>	300	0,733 $\pm$ 0,377	1,213 $\pm$ 0,370
<i>Neighborhood</i>	500	0,604 $\pm$ 0,222	1,070 $\pm$ 0,309
<i>Neighborhood</i>	1000	0,630 $\pm$ 0,212	1,190 $\pm$ 0,209
<i>Neighborhood</i>	1500	0,598 $\pm$ 0,053	1,151 $\pm$ 0,130
<i>Neighborhood</i>	2000	0,607 $\pm$ 0,100	1,194 $\pm$ 0,164
<i>Popularity</i>	100	0,168 $\pm$ 0,189	4,661 $\pm$ 1,433
<i>Popularity</i>	150	0,046 $\pm$ 0,141	5,295 $\pm$ 1,291
<i>Popularity</i>	300	0,023 $\pm$ 0,039	6,075 $\pm$ 0,933
<i>Popularity</i>	500	0,038 $\pm$ 0,039	5,224 $\pm$ 0

Tabela 5.17: Gap médio entre valor da relaxação linear e valor da solução ótima para instâncias solucionadas pelos modelos (DIB-F) e (DDI).

ao contrário do que ocorreu nas instâncias esparsas. Um primeiro aspecto que exerce influência sobre a performance dos modelos é a quantidade de restrições, que é muito maior em (DDI), como podemos verificar na Tabela 5.19. Assintoticamente, para instâncias densas, (DIV) e (DDI) têm $\Theta(\|v\|_0 + |B|)$ e $\Theta(|B| + \|v\|_0|B|)$ restrições, respectivamente. Ou seja, (DDI) teria aproximadamente $|B|$ vezes mais restrições que (DIV), na teoria. Na prática, a escala de diferença é muito menor, mas a quantidade de restrições de (DDI), mesmo após o *presolve*, continua sendo muito maior. Mesmo com uma diferença tão grande na quantidade de restrições, o tempo de execução de (DDI) foi competitivo, indicando que se houvesse uma outra variante com menos restrições, talvez esta tivesse obtido o melhor desempenho para as instâncias densas.

Modelo	Tamanho (n)					
	100		150		200	
	Qtd	Avg $\pm$ Std	Qtd	Avg $\pm$ Std	Qtd	Avg $\pm$ Std
DIB	9	1.855,61 $\pm$ 228,75s	-	-	-	-
DIB-F	6	3.219,94 $\pm$ 349,52s	-	-	-	-
DD	4	1.027,24 $\pm$ 1.183,16s	-	-	-	-
DD-F	4	1.266,19 $\pm$ 1.245,59s	-	-	-	-
DIV	10	12,65s $\pm$ 4,41s	10	44,34s $\pm$ 7,86s	10	93,22s $\pm$ 12,67s
DIV-F	10	25,58s $\pm$ 9,22s	10	102,65s $\pm$ 19,05s	10	273,83s $\pm$ 32,88s
DDI	10	13,92s $\pm$ 5,25s	10	68,12s $\pm$ 11,55s	10	219,69s $\pm$ 14,12s

Tabela 5.18: Número de instâncias densas solucionadas e tempo de execução para todos os modelos.

Tamanho (n)	DIV		DDI	
	Vars $\pm$ Std	Constrs $\pm$ Std	Vars $\pm$ Std	Constrs $\pm$ Std
100	9.854,40 $\pm$ 35,58	29.392,70 $\pm$ 79,25	3.406,70 $\pm$ 416,08	141.972,80 $\pm$ 21.596,57
150	22.009,70 $\pm$ 92,42	66.031,40 $\pm$ 215,95	8.064,30 $\pm$ 751,13	563.953,70 $\pm$ 64.420
200	39.217,10 $\pm$ 306,52	117.809,10 $\pm$ 621,78	13.950,70 $\pm$ 856,57	1.327.896,90 $\pm$ 102.239,15

Tabela 5.19: Número médio de variáveis inteiras e restrições após o *presolve* para instâncias densas solucionadas pelos modelos (DIV) e (DDI).

Mesmo com mais restrições, se analisarmos o Gap médio, na Tabela 5.20, vemos que a relaxação linear de (DDI) para as instâncias densas é muito boa, e por conta disso talvez (DDI) pudesse ter obtido um resultado melhor que (DIV), já que o esforço para se encontrar uma solução ótima foi reduzido devido ao excelente limitante. E de fato teria obtido se a maior parte do tempo de execução de (DDI) não tivesse sido gasta no *presolve* e na resolução da relaxação linear do nó raiz. Na Tabela 5.21 encontramos o tempo médio de execução (**Avg**) dos modelos (DIV) e (DDI) descontando o tempo gasto no *presolve* e na relaxação linear. Podemos notar que, nesse cenário, o tempo médio de execução de (DDI) foi sempre menor que o de (DIV), e que de fato a maior quantidade de restrições de (DDI) fez com que uma porcentagem significativa do tempo fosse gasta em outras etapas, elevando o tempo total de execução.

Tamanho (n)	DIV	DDI
	Gap (%) $\pm$ Std	Gap (%) $\pm$ Std
100	0,352 $\pm$ 0,100	0,052 $\pm$ 0,032
150	0,292 $\pm$ 0,087	0,055 $\pm$ 0,046
200	0,212 $\pm$ 0,021	0,038 $\pm$ 0,016

Tabela 5.20: Gap médio entre valor da relaxação linear e valor da solução ótima para instâncias densas solucionadas pelos modelos (DIV) e (DDI).

Tamanho (n)	DIV	DDI
	Avg $\pm$ Std	Avg $\pm$ Std
100	10,08s $\pm$ 4,39s	4,23s $\pm$ 2,29s
150	32,13s $\pm$ 8,24s	23,54s $\pm$ 10,31s
200	64,08s $\pm$ 13,72s	59,94s $\pm$ 13,04s

Tabela 5.21: Tempo médio de execução após *presolve* e relaxação linear do nó raiz para instâncias densas solucionadas pelos modelos (DIV) e (DDI).

Capítulo 6

Conclusão

Este trabalho foi motivado pelo aspecto prático do Problema da Compra Mínima e por sua investigação ser escassa na literatura até então, e propôs avanços na área por meio de técnicas de Programação Linear Inteira. Foram propostas 4 formulações para o Problema da Compra Mínima, 1 de Programação Linear Inteira e 3 de Programação Linear Inteira Mista. O modelo (DIB), de PLI, foi construído com base na formulação de Aggarwal et al. [1] para o Problema da Compra Máxima, adicionando restrições específicas que garantem a escolha do item viável de menor preço para cada comprador, respeitando as exigências do problema. Já o modelo (DD), de PLIM, inspirado na formulação de Heilporn et al. [18] para o Problema de Precificação Livre de Inveja, utiliza uma abordagem mais direta para determinar as alocações e os preços. A partir de (DD), construímos outro modelo, (DIV), com menos variáveis e restrições, e, por fim, apresentamos o modelo (DDI), também de PLIM, com ainda menos variáveis e que infere o preço pago por um comprador a partir dos preços pagos pelos outros compradores. Apresentamos restrições para fortalecer os modelos (DIB), (DD) e (DIV), com o objetivo de reduzir o tempo de execução e aumentar a quantidade de instâncias solucionadas, e os chamamos (DIB-F), (DD-F) e (DIV-F). Apresentamos um teorema que prova que basta considerar um subconjunto de preços para cada item, e esse subconjunto são as valorações dadas àquele item. Por meio do teorema, reduzimos o número de variáveis e restrições dos modelos, tornando possível sua utilização na prática, principalmente para instâncias esparsas.

Os testes realizados demonstraram que a adição das restrições foi benéfica para os modelos, principalmente para os casos de (DIB-F), onde o tempo de execução foi reduzido, e de (DIV-F), onde o tempo de execução também foi reduzido e a quantidade de instâncias solucionadas aumentou. Para instâncias esparsas, como foram as instâncias das classes *Characteristics*, *Neighborhood* e *Popularity*, o modelo (DIB-F) se demonstrou mais eficiente, solucionando praticamente todas as instâncias com uma excelente relaxação. Para instâncias densas, o modelo (DIV) obteve o menor tempo de execução, conseguindo solucionar todas as instâncias. O modelo (DDI) apresentou um bom equilíbrio entre instâncias esparsas e densas, sendo o melhor modelo de PLIM para instâncias esparsas, em termos de tempo de execução e quantidade de instâncias solucionadas, e também conseguindo solucionar todas as instâncias completas com tempos de execução não muito distantes dos obtidos por (DIV).

Apesar de suas contribuições, o estudo apresenta limitações. Os experimentos com-

putacionais foram realizados em um subconjunto de tipos de instâncias, deixando espaço para explorar cenários mais diversos, como com quantidades diferentes de compradores e itens. Técnicas de pré-processamento podem ser utilizadas para identificar preços ótimos e alocações de forma estática, reduzindo o tamanho das instâncias. Além disso, ainda há espaço para a exploração de métodos híbridos que integrem abordagens heurísticas e exatas, e também para uma investigação mais aprofundada sobre a natureza do problema, que leve à ideia de cortes eficientes que possam ser utilizados em uma abordagem de *Branch-and-Cut*, por exemplo.

Referências Bibliográficas

- [1] Gagan Aggarwal, Tomás Feder, Rajeev Motwani, and An Zhu. Algorithms for multi-product pricing. In *Proceedings of the 31th International Colloquium on Automata, Languages and Programming*, pages 72–83, 2004.
- [2] Patrick Briest and Piotr Krysta. Buying cheap is expensive: Hardness of non-parametric multi-product pricing. In *Proceedings of the 18th Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 716–725, 2007.
- [3] Herminia I. Calvete, Concepción Domínguez, Carmen Galé, Martine Labbé, and Alfredo Marín. The rank pricing problem: models and branch-and-cut algorithms. *Computers & operations research*, 105:12–31, 2019.
- [4] Parinya Chalermsook, Julia Chuzhoy, Sampath Kannan, and Sanjeev Khanna. Improved hardness results for profit maximization pricing problems with unlimited supply. In *Proceedings of the 15th International Workshop APPROX*, pages 73–84, 2012.
- [5] Parinya Chalermsook, Bundit Laekhanukit, and Danupon Nanongkai. Independent set, induced matching, and pricing: Connections and tight (subexponential time) approximation hardnesses. In *Proceedings of the 54th Annual Symposium on Foundations of Computer Science*, pages 370–379, 2013.
- [6] Ning Chen and Xiaotie Deng. Envy-free pricing in multi-item markets. In *Proceedings of the 37th International Colloquium Conference on Automata, Languages and Programming: Part II*, pages 418–429, 2010.
- [7] George B. Dantzig. Maximization of a linear function of variables subject to linear inequalities. *Activity analysis of production and allocation*, 13:339–347, 1951.
- [8] Gregory Dobson and Shlomo Kalish. Heuristics for pricing and positioning a product-line using conjoint and cost data. *Management Science*, 39:160–175, 1993.
- [9] Concepción Domínguez. Formulations for the capacitated rank pricing problem. In *Proceedings of the European Chapter on Combinatorial Optimization, XXXII Conference*, page 28, 2019.
- [10] Concepción Domínguez, Martine Labbé, and Alfredo Marín. The rank pricing problem with ties. *European Journal of Operational Research*, 294:492–506, 2021.

- [11] Concepción Domínguez, Martine Labbé, and Alfredo Marín. Mixed-integer formulations for the capacitated rank pricing problem with envy. *Computers & Operations Research*, 140:105664, 2022.
- [12] Cristina G. Fernandes, Carlos E. Ferreira, Álvaro J. P. Franco, and Rafael C. S. Schouery. The envy-free pricing problem, unit-demand markets and connections with the network pricing problem. *Discrete Optimization*, 22:141–161, 2016.
- [13] Cristina G. Fernandes and Rafael C. S. Schouery. Approximation algorithms for the max-buying problem with limited supply. *Algorithmica*, 80:2973–2992, 2018.
- [14] David Gale, Harold W. Kuhn, and Albert W. Tucker. Linear programming and the theory of games. *Activity analysis of production and allocation*, 13:317–329, 1951.
- [15] Michael R. Garey and David S. Johnson. *Computers and intractability*, volume 174. W. H. Freeman, 1979.
- [16] Venkatesan Guruswami, Jason D. Hartline, Anna R. Karlin, David Kempe, Claire Kenyon, and Frank McSherry. On profit-maximizing envy-free pricing. In *Proceedings of the 16th Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 1164–1173, 2005.
- [17] Géraldine Heilporn, Martine Labbé, Patrice Marcotte, and Gilles Savard. A parallel between two classes of pricing problems in transportation and marketing. *Journal of Revenue and Pricing Management*, 9:110–125, 2010.
- [18] Géraldine Heilporn, Martine Labbé, Patrice Marcotte, and Gilles Savard. A polyhedral study of the network pricing problem with connected toll arcs. *Networks: An International Journal*, 55:234–246, 2010.
- [19] Narendra Karmarkar. A new polynomial-time algorithm for linear programming. *Combinatorica*, 4:373–395, 1984.
- [20] Bernhard Korte and Jens Vygen. *Combinatorial Optimization: Theory and Algorithms*, pages 71–97. Springer, 2008.
- [21] Ailsa H. Land and A. G. Doig. An automatic method of solving discrete programming problems. *Econometrica*, 28:497–520, 1960.
- [22] Tor G. J. Myklebust, Malcolm A. Sharpe, and Levent Tunçel. Efficient heuristic algorithms for maximum utility product pricing problems. *Computers & Operations Research*, 69:25–39, 2016.
- [23] Shmuel S. Oren, Stephen A. Smith, and Robert B. Wilson. Pricing a product line. *Journal of Business*, 57:S73–S99, 1984.
- [24] Shmuel S. Oren, Stephen A. Smith, and Robert B. Wilson. Multi-product pricing for electric power. *Energy Economics*, 9:104–114, 1987.

- [25] Paat Rusmevichientong, Benjamin Van Roy, and Peter W. Glynn. A nonparametric approach to multiproduct pricing. *Operations Research*, 54:82–98, 2006.
- [26] Romy Shioda, Levent Tunçel, and Tor G. J. Myklebust. Maximum utility product pricing models and algorithms based on reservation price. *Computational Optimization and Applications*, 48:157–198, March 2011.
- [27] Stephen A. Smith. New product pricing in quality sensitive markets. *Marketing Science*, 5:70–87, 1986.